Bachelor's Thesis

Multisensor Data Acquisition and Real-Time Fusion for Neurotherapy Using EMG and Accelerometer Sensor On an Embedded Linux System with connected Patient Monitor

Waleed Rajab

Feb 24, 2026

Submitted to the University of Freiburg
IMTEK – Department of Microsystems Engineering
Laboratory for Electrical Instrumentation and Embedded Systems

universität freiburg

University of Freiburg
IMTEK – Department of Microsystems Engineering
Laboratory for Electrical Instrumentation and Embedded Systems

Author Waleed Rajab,
Matriculation Number: 4772281

Editing Time Dec 11 - Feb 24, 2026

Examiners Prof. Dr. Thomas Stieglitz,
IMTEK – Department of Microsystems Engineering
Laboratory for Electrical Instrumentation and Embedded Systems

Supervior Christian Stolle,
Neuer Messplatz 3
CorTec GmbH

Declaration I hereby declare, that I am the sole author and composer of this Thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I hereby also declare, that my Thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Freiburg, Feb 24, 2026



Place, Date

Signature

Declaration of using AI and Apps

During the preparation of this thesis, I used generative AI tools to support the research process and for editorial assistance. In particular, AI was used for preliminary literature discovery and to improve the language, style, clarity of the text, and for coding. The tools used were NotebookLM [1], DeepL [2], [3], ChatGPT [4], GitHub Copilot [5], [6], and draw.io [7]. The conceptual structure, argumentation, and final content of this thesis are my own original work.

Confidentiality Notice

This Bachelor's thesis entitled:

"Multisensor Data Acquisition and Real-Time Fusion for Neurotherapy Using EMG and Accelerometer Sensors on an Embedded Linux System with Connected Patient Monitor"

contains confidential information and internal data of CorTec GmbH. It may be used only for examination purposes. Without the prior written permission of CorTec GmbH, it must not be shared with third parties or published, in whole or in part. The thesis may be provided to university staff involved in the examination and to the examiners. These persons are also required to keep the information confidential. If required, the thesis may also be used in legal proceedings related to the examination under the applicable administrative rules.

Abstract

Multisensor pipelines in wearable and medical systems are limited not only by signal processing quality, but also by the timing of the complete data path. This thesis analyzes the end-to-end timing of a hybrid setup that combines a Bluetooth® Low Energy (BLE)-based surface Electromyography (sEMG) stream with a high-rate Accelerometer (ACC) stream read via Inter-Integrated Circuit (I^2C) on a Raspberry Pi (Pi). The fused data is then forwarded to a tablet application. Two fusion modes are implemented and compared. The first mode is an event-driven forwarder that outputs data whenever a new sample arrives. The second mode is a feature-based fusion that runs with a fixed internal rate of 1 kHz. In the experiments, the ACC was configured for 833 Hz, but the effective rate at the Hardware Abstraction Layer (HAL) input was lower (mean 650.8 Hz, median 647 Hz, 21.9% drop). This reduction was mainly caused by host-side I^2C read-out and operating-system scheduling overhead. The Electromyography (EMG) device is specified for 10 Hz. However, because two samples were batched in each BLE notification, the effective rate at the HAL input was 5.0 Hz. As a result, the forwarder fusion output rate was mainly driven by the ACC stream (mean 659.3 Hz, median 656 Hz). In contrast, the feature-based fusion produced a stable 1 kHz output rate even when the input rates were irregular and reduced. Processing delays inside the HAL were very small (median 0.026 ms for ACC and 0.137 ms for EMG). This indicates that most timing variations come from earlier steps, especially wireless transmission and host-side acquisition. The average payload bandwidth at the final fusion output was 168.9 kbit/s for the forwarder and 256 kbit/s for the feature-based approach. Finally, hardware tests showed that a Pi Zero 2 W could not reliably maintain the required parallel BLE links, while a Pi 5 remained stable for the full measurement duration.

Contents

1 Introduction **1**

1.1 Motivation 2

1.1.1 Problem Description 2

1.1.2 Research question 3

2 Theoretical Background **5**

2.1 Combined sensing in wearable systems 5

2.1.1 Surface electromyography 5

2.1.2 Accelerometer 6

2.2 gRPC communication model 6

2.3 Real-time feature computation 7

2.3.1 Acceleration magnitude as a scalar motion measure 7

2.3.2 Root Mean Square for EMG 8

2.3.3 Rate throttling 8

2.3.4 Sample and hold 9

2.3.5 Sliding Window with fixed step size 9

2.3.6 Variance of acceleration magnitude 11

2.3.7 Effective rate, delay, jitter, and bandwidth 11

3 State of the Art **13**

3.1 sEMG and accelerometer 13

3.1.1 Multisensor fusion and timing 13

3.2 Bluetooth Low Energy (BLE) 13

3.3 Protocol Buffer (Protobuf) 14

3.4 gRPC Remote Procedure Calls (gRPC) 14

3.5 Alignment 14

4 Implementation **17**

4.1 System overview 17

4.2 Hardware and runtime environment 19

4.3 Internal transport and buffering 19
4.4 Sensor acquisition 20
4.4.1 ACC Manager (HAL) 21
4.4.2 EMG Manager (HAL) 21
4.5 Protocol Buffers in HAL interface 21
4.6 Fusion manager 22
4.6.1 Alignment 22
4.6.2 Forwarder fusion 23
4.6.3 Feature-based fusion 24
4.6.4 Fused output stream 24
4.7 BLE Peripheral 24
4.8 Flutter Application 25
4.9 Timing measurements 25
4.10 Data analysis 26

5 Experimental Setup 27
5.1 Setup 27
5.2 Configuration 28
5.3 Software and services 28
5.4 Software libraries 29
5.5 Tablet Application 30
5.6 Measurement Session 30

6 Results and Discussion 31
6.1 Live visualization on the tablet 31
6.2 Hardware feasibility 32
6.3 Effective rate 32
6.3.1 Accelerometer 33
6.3.2 EMG 34
6.3.3 Forwarder-based fusion 35
6.3.4 Feature-based fusion 37
6.4 Jitter, delay and bandwidth 38
6.4.1 Jitter and Effective Period 38
6.4.2 Delay 43
6.4.3 Bandwidth 45

7 Conclusion **47**
7.1 Outlook 49

8 Acknowledgments **51**

Bibliography **53**

A Supplementary Material **A**
A.1 Implant unit . . . . . A

Symbol Definitions

Symbol	Description	Unit
B	Average application-layer bandwidth	bit/s
Δ	Step size of the fusion clock	ms
σ_m^2	Variance of acceleration magnitude within the window	$\mathrm{m}^2/\mathrm{s}^4$
a_x	Acceleration along x-axis	m/s^2
a_y	Acceleration along y-axis	m/s^2
a_z	Acceleration along z-axis	m/s^2
d	Delay between two measurement points	ms
m	Acceleration magnitude	m/s^2
r_{eff}	Effective rate	Hz
t_k	Fusion tick time, $t_k = t_0 + k\Delta$	s

Acronyms

Acronyms	Description
ACC	Accelerometer
ADV	Advertising
ARM	Advanced RISC Machines
ATT	Attribute Protocol (BLE)
BLE	Bluetooth® Low Energy
D-Bus	Desktop Bus
EMG	Electromyography
FES	Functional Electrical Stimulation
FIFO	First In First Out
GATT	Generic Attribute Profile
gRPC	Generell Remote Procedure Call
HAL	Hardware Abstraction Layer
HTTP	Hypertext Transfer Protocol
I^2C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
MTU	Maximum Transmission Unit
PHY	Physical Layer
Pi	Raspberry Pi
ProtoBLE	ProtoBuffer over BLE/GATT
Protobuf	Protocol Buffers
RMS	Root Mean Square
RPC	Remote Procedure Call
SBC	Single Board Computer
sEMG	surface Electromyography
TLS	Transport Layer Security

Figure Index

2.1 gRPC over an HTTP/2 connection using server-side streaming. . 7
2.2 Sample and hold principle. 9
2.3 Sliding-window fusion with window length W and step size Δ, ($\Delta < W$) 10

4.1 System overview. 17
4.2 Broadcaster-based internal transport with per-subscriber queues. . 20
4.3 Samples alignment 23
4.4 Measurement points used to evaluate delay, jitter, and bandwidth in the streaming fusion pipeline. 26

5.1 The setup. 28

6.1 Flutter/Dart Application (*Feature-based fusion*). 32
6.2 Effective Accelerometer rate at HAL entry. 34
6.3 Effective EMG rate at HAL entry (two samples per notification). 35
6.4 Effective rate at the forwarder-based fusion output. 36
6.5 Effective rate at the feature-based fusion output. 37
6.6 Jitter at the accelerometer HAL entry. 39
6.7 Jitter at the EMG HAL entry. 40
6.8 Jitter at the Fusion Forwarder output. 41
6.9 Jitter at the Fusion feature output. 42
6.10 Delay within the accelerometer HAL measured from `ENTRY` to `OUT`. 43
6.11 Delay within the EMG HAL measured from `ENTRY` to `OUT`. 44

A.1 Implant unit with connected multi-electrode arrays of the CorTec Brain Interchange System. . . . . B

Chapter 1

Introduction

Motor impairments of the upper limb often occur after neurological injuries such as stroke and can strongly reduce independence in daily life [8]–[10]. A key movement for functional arm use is elbow flexion, because many everyday actions require bringing the hand toward the body, for example eating, dressing, or holding an object close [8]. Therefore, restoring or supporting elbow flexion can already improve arm function, even if fine hand movements remain limited [9]. Different approaches are used to improve upper-limb function. These include medication, physiotherapy with task-oriented exercises, robotic assistance, and electrical stimulation [9]. Stimulation is of special interest because it can directly activate muscles by applying electrical pulses to peripheral nerves or muscles [11], [12]. If stimulation is controlled in a suitable way, it can support functional movements and may also support recovery through repeated activation that is linked to the patient's intention [13]–[15]. This thesis is motivated by an implant-assisted stimulation concept that targets the musculocutaneous nerve to restore or support biceps activation and thereby enable elbow flexion [12], [16]. Such approaches are motivated by the potential for more direct and selective activation compared to purely external electrical stimulation via skin electrodes, and they can support stable long-term interfacing [17]. The implant communicates with an external device that provides power and control. The external device contains an accelerometer, and muscle activity is additionally recorded with a sEMG sensor. sEMG and inertial sensing are often combined to improve robustness compared to single-modality approaches [18]. EMG-driven or intention-based stimulation strategies, including from the opposite limb controlled Functional Electrical Stimulation (FES), have been studied to support upper-limb recovery and functional use after acquired brain injury and stroke.[19]–[21]. However, using multiple sensors also increases system complexity. Several data streams must be

acquired, transmitted, aligned in time, and processed before the outputs can be used for control [22]. Wearable and embedded sensor systems are widely used in medical and assistive applications because they allow continuous measurement close to the user while keeping the setup compact and mobile [23], [24]. In practice, such systems often combine different technologies. Wireless links such as BLE reduce cabling and are flexible, but they can show variable timing due to limited transmission opportunities, retransmissions, and operating system scheduling [25]–[27]. Wired interfaces usually provide more stable timing, but they still depend on polling rates, driver behavior, and CPU load [28], [29]. If wireless and wired links are combined, their timing differences can accumulate and lead to irregular update intervals, bursts of data, or effective sampling rates that differ from the configured rates [25]–[27]. This timing behavior is important for real-time fusion and control. Many methods rely on sliding windows or fixed step sizes to compute features and generate stable outputs [18], [30]. If samples arrive irregularly, windows may cover uneven time spans. In addition, buffering may be needed to align streams, which increases latency [22]. If the system cannot process data fast enough, queues can grow and old samples may be dropped, which changes the content of the fused output. In live displays, these effects can appear as unstable plots, delayed responses, or a mismatch between expected and observed update rates. The system studied in this thesis combines an sEMG sensor transmitted via BLE with a high-rate accelerometer stream acquired through a wired interface. Both streams are received on a Pi, handled by hardware abstraction layers, and processed by a fusion component that produces outputs for a tablet application. Because the system comprises multiple software layers and execution contexts, its timing behavior is evaluated by adding timestamps at defined measurement points along the pipeline. The resulting traces enable a statistical analysis of processing and transmission delays of each component involved in the pipeline.

1.1 Motivation

1.1.1 Problem Description

Assistive implants can support impaired arm function by detecting movement intention and applying targeted stimulation. Wearable sensors such as sEMG and inertial sensing provide complementary information, but they must be synchronized and processed in real time. In practice, delays and jitter can misalign both

signals and reduce the reliability of fusion and control. To reduce data rate and processing load, many systems compute simple features early instead of streaming minimally processed signals. This can simplify the pipeline, but it may remove useful information and can limit precise time alignment. In this thesis, the system combines a wired high-rate accelerometer stream with a sEMG stream over BLE, both processed on a Pi and sent to a tablet.

1.1.2 Research question

Is it possible to combine sensor readings on a Pi from a wired accelerometer (833 Hz) and a BLE-connected sEMG sensor (10 Hz) such that the required information can be transmitted to an external system at an effective rate of 1 kHz?

Chapter 2

Theoretical Background

This chapter summarizes the foundations of real-time multisensor acquisition and fusion using sEMG and accelerometer data. It outlines key signal characteristics, common embedded preprocessing steps, and constraints of wireless transmission, in particular BLE. The chapter further introduces time alignment and window-based fusion, and defines delay, jitter, and effective rate as metrics for evaluating timing performance.

2.1 Combined sensing in wearable systems

Wearable sensors operate close to the body and are affected by motion, changing contact conditions, and noise. To improve robustness, systems often combine multiple modalities instead of relying on a single sensor [31]. A common combination is sEMG and acceleration: sEMG reflects muscle activation, while acceleration captures the resulting movement.

2.1.1 Surface electromyography

An sEMG sensor records electrical activity produced by muscles using electrodes placed on the skin. It provides information about activation strength and timing, but it is sensitive to practical factors such as electrode placement, skin contact quality, motion artifacts, and electromagnetic interference [32]. Therefore, sEMG is often processed in short time windows or converted into more stable measures. In this project, sEMG is acquired with the ANR M40 sensor, which uses a two-electrode measurement without a reference lead [33]. The M40 internally samples at 4800 Hz, processes the data internally using proprietary data processing, transmits the EMG level via BLE notifications every 100 ms (10 Hz) .

2.1.2 Accelerometer

A 3-axis accelerometer measures linear acceleration along three orthogonal axes (a_x, a_y, a_z). In wearable use, the signal contains dynamic components caused by motion and a static component caused by gravity. Compared to sEMG, accelerometer data is typically easier to acquire consistently, but it depends on sensor orientation and placement [34]. Therefore, many systems use derived measures such as the acceleration magnitude to describe overall motion intensity. In this project, acceleration is measured with the STMicroelectronics LSM6DSOX, a 6-axis Inertial Measurement Unit (IMU) that includes a 3-axis accelerometer [35]. The accelerometer supports selectable full-scale ranges of $\pm 2/\pm 4/\pm 8/\pm 16\,g$ and configurable output data rates up to 6664 Hz, which allows operating at high sampling rates when needed.

2.2 gRPC communication model

Generell Remote Procedure Call (gRPC) is a framework that allows a client to call functions on a remote server through a locally generated stub [36]. The interface and message formats are defined in `.proto` files using Protocol Buffers, which provide compact binary encoding and strongly typed data structures [37]. gRPC typically runs over Hypertext Transfer Protocol (HTTP)/2 [38]. For continuous sensor pipelines, server-side streaming is especially useful: after a client sends a single subscription request, the server keeps the connection open and continuously transmits messages as new samples become available [39]. This principle is illustrated in Fig. 2.1. While gRPC makes structured streaming easier, it does not guarantee hard real-time behavior. Latency and jitter still depend on factors such as scheduling, buffering, and network conditions. For this reason, streamed sensor messages usually include timestamps to support ordering and reliable temporal alignment across modalities [40].

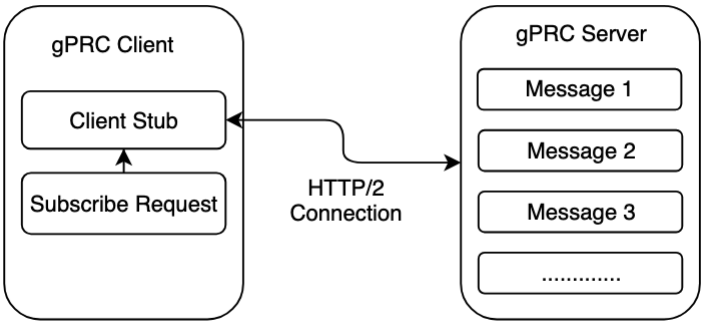


Figure 2.1: gRPC over an HTTP/2 connection using server-side streaming.

2.3 Real-time feature computation

In embedded real-time systems, there is a common trade-off between transmitting raw signals and sending compact features. Features reduce data volume and can make downstream processing easier and more stable. However, they also remove signal detail that may still be useful for later fusion or classification. For this reason, when comparing a feature-based pipeline with a minimally processed alternative, it is important to be clear about what the chosen features capture and what they leave out.

2.3.1 Acceleration magnitude as a scalar motion measure

A common scalar measure derived from 3D acceleration is the Euclidean magnitude [41]:

$$\mathrm{MAG}(x) = \sqrt{a_x^2 + a_y^2 + a_z^2}. \tag{2.1}$$

It summarizes overall motion intensity in a largely orientation-insensitive way and reduces the 3-axis signal to a single stream. However, it removes directional and axis-specific information, so different motions can produce similar magnitude traces. Therefore, magnitude is useful for coarse intensity estimation but may be insufficient for tasks that require motion direction or coordination patterns.

2.3.2 Root Mean Square for EMG

A common EMG feature is the Root Mean Square (RMS) computed over a window of N samples [42]:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2}. \tag{2.2}$$

The RMS summarizes activation intensity within the window and is more stable than the raw waveform, which is useful for real-time processing. Its main limitation is information loss: it smooths fast changes and reduces waveform detail. In addition, RMS refers to a time interval and can introduce an effective latency.

2.3.3 Rate throttling

In a combined sensors real-time pipeline, a sensor may produce samples faster than the common processing grid used for fusion. A practical example is an accelerometer configured at a high output data rate (around 1.6 kHz), while the fusion pipeline operates on a 1 kHz time base. In this case, the stream can be rate throttled to the target rate: within each 1 ms interval, at most one sample is forwarded, and additional samples are skipped. Rate throttling is a system-level measure. It leaves the sensor sampling unchanged, but limits how many samples are forwarded through processing and communication, as a result keeping load and latency bounded and ensuring that the sensor stream does not exceed the fusion grid.

2.3.4 Sample and hold

In real-time systems, different sensors do not always deliver data at the same time. Short gaps can happen because of buffering, scheduling, or transport jitter. Then, a fusion step may not receive a new value from one modality at a certain moment. To keep the output stable, many systems use *sample-and-hold*. If no new sample is available, the system reuses the last valid value until a new sample arrives. This avoids sudden drops in the output. In practice, the hold time is often limited [43]. If the last value is too old, it is treated as invalid.

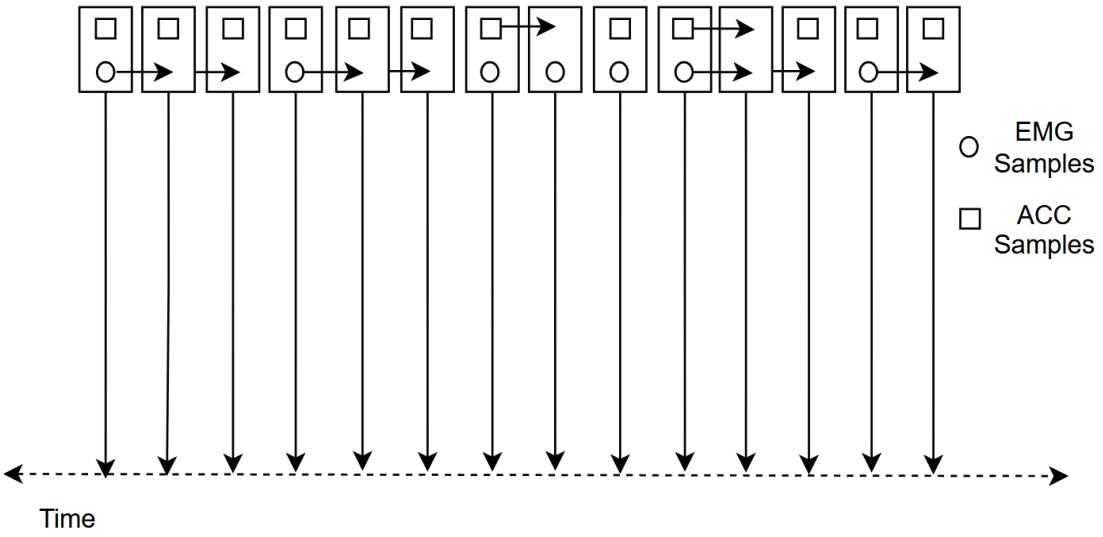


Figure 2.2: Sample and hold principle.

2.3.5 Sliding Window with fixed step size

For continuous sensing, the system receives two time-stamped sample streams: EMG samples and ACC samples. Instead of processing each incoming sample individually, both signals are evaluated in short time segments (windows) of fixed duration W [44]–[46]. A discrete *fusion clock* defines the evaluation instants t_k in constant steps Δ, where $k = 0, 1, 2, \ldots$ denotes the tick (window) index:

$$t_k = t_0 + k\Delta.$$

At each tick t_k, a window W_k is formed by collecting all samples whose timestamps fall into the most recent interval of length W:

$$W_k = \{\, x_i \mid t_i \in [t_k - W,\ t_k] \,\}.$$

In other words, W_k contains the most recent EMG and ACC samples that lie within the same time interval. A processing function $g(\cdot)$ is then applied to the window to produce one fused output for this tick:

$$f[k] = g(W_k).$$

Here, $f[k]$ denotes the fused result produced at tick k. The function $g(\cdot)$ represents the concrete processing performed on the window. The step size Δ controls the output rate, one result every Δ seconds. If a denser output is required, choosing $\Delta < W$ makes consecutive windows overlap and reuse samples across ticks, increasing temporal resolution at the cost of higher computational load. For $\Delta = W$, windows are adjacent, and for $\Delta > W$ gaps can be happens.

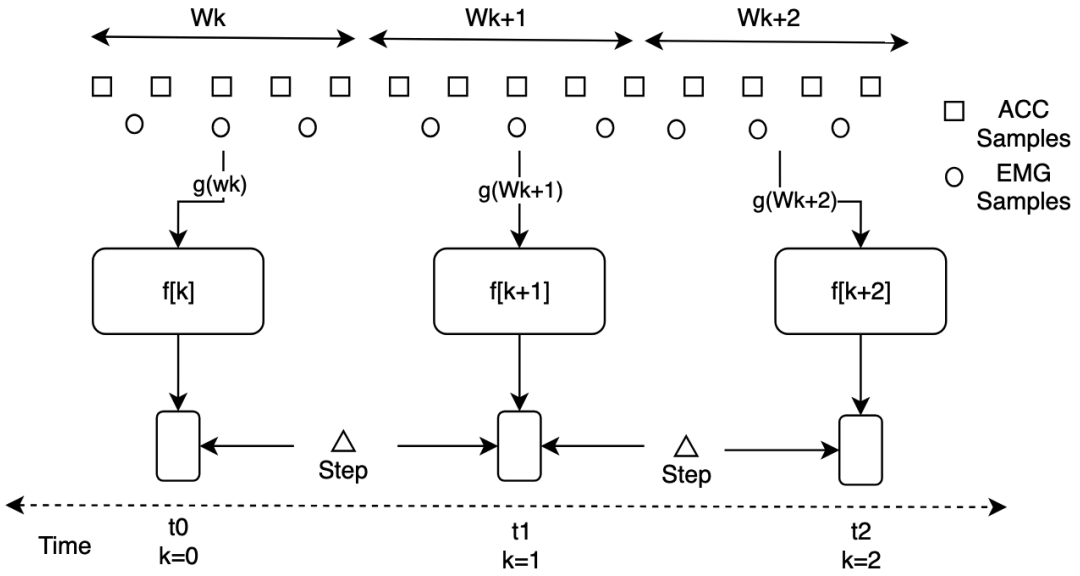


Figure 2.3: Sliding-window fusion with window length W and step size Δ, ($\Delta < W$) .

2.3.6 Variance of acceleration magnitude

Besides the acceleration magnitude Eq. (2.1), the short-term variability of motion can be described by the variance of the magnitude within the sliding window defined in Section 2.3.5. Let $\{m_i\}_{i=1}^{N_k}$ be the N_k magnitude samples in the current window $[t_k - W,\, t_k]$. The window variance is

$$\sigma_k^2 \;=\; \frac{1}{N_k}\sum_{i=1}^{N_k} m_i^2 \;-\; \left(\frac{1}{N_k}\sum_{i=1}^{N_k} m_i\right)^2 . \tag{2.3}$$

A small σ_k^2 indicates nearly constant magnitudes (steady motion), whereas a larger σ_k^2 indicates more dynamic movement [47]. This form is efficient for real-time use because it can be computed from running sums over the window [48].

2.3.7 Effective rate, delay, jitter, and bandwidth

To characterize the timing behavior of the streaming pipeline, timestamps are recorded at selected measurement points along the processing chain. For item i (samples or fused output), we denote the timestamps at the relevant stages as $t_i^{\mathrm{HAL,in}}$ (HAL input), $t_i^{\mathrm{HAL,out}}$ (HAL output), and $t_i^{\mathrm{F,out}}$ (fusion output). The index i is used to match timestamps that belong to the same logical item.

Effective rate

The effective rate describes how many items per second are observed at a given measurement point. For timestamps $t_1^X, \ldots, t_N^X$, it is computed as

$$r_{\mathrm{eff}}^X = \frac{N-1}{t_N^X - t_1^X} \quad \text{(Hz)}.$$

It is evaluated at the HAL input, HAL output, and fusion output to detect rate changes introduced by the pipeline.

Delay

Delay is the time difference between two measurement points for the same item:

$$d_i^{X\to Y} = t_i^Y - t_i^X.$$

Two delays are measured:

$$d_i^{\mathrm{HAL,in}\to\mathrm{HAL,out}} = t_i^{\mathrm{HAL,out}} - t_i^{\mathrm{HAL,in}}, \qquad d_i^{\mathrm{HAL,in}\to\mathrm{F,out}} = t_i^{\mathrm{F,out}} - t_i^{\mathrm{HAL,in}}.$$

Here, $d_i^{\mathrm{HAL,in}\to\mathrm{HAL,out}}$ captures the processing and buffering delay inside the HAL stage, while $d_i^{\mathrm{HAL,in}\to\mathrm{F,out}}$ describes the end-to-end delay from sensor arrival at the HAL to the availability of the fused output.

Jitter

Jitter describes short-term variations in the timing of an update stream, i.e., how much the time between consecutive updates changes. For a timestamp sequence $t_1, \ldots, t_N$, the inter-arrival intervals are

$$\Delta t_i = t_i - t_{i-1} \qquad (i = 2, \ldots, N).$$

Jitter can be described by how much these intervals $\{\Delta t_i\}$ vary, for example using the range, the standard deviation, or the middle 50% range (IQR) [49],

$$\mathrm{IQR}(\Delta t) = Q_{75}(\Delta t) - Q_{25}(\Delta t).$$

Small values indicate stable timing, while larger values indicate stronger timing fluctuations.

Bandwidth

Bandwidth is evaluated at the final fusion output stream. For payload sizes s_i (bytes) observed over the interval $[t_1, t_N]$, the average application-layer throughput is

$$B = \frac{8 \sum_{i=1}^{N} s_i}{t_N - t_1} \quad \text{(bit/s)}.$$

This measure reflects payload throughput at the application layer and excludes lower-layer protocol overhead.

Chapter 3

State of the Art

3.1 sEMG and accelerometer

Wearable systems often combine sEMG and inertial sensing because both modalities describe different aspects of movement 2.1.1 2.1.2. sEMG reflects neuromuscular activation, while acceleration captures motion dynamics. Recent reviews summarize that combining these modalities can improve robustness compared to single sensor approaches, but it also increases requirements on synchronization and consistent processing across streams [18], [50].

3.1.1 Multisensor fusion and timing

Real-time fusion depends on maintaining a consistent temporal relation between modalities across the entire pipeline. Differences in sampling rates, buffering, and transport delays can shift the alignment between streams. Therefore, modern systems typically use timestamps captured close to acquisition and well-defined buffering/alignment rules, ensuring that fused outputs correspond to a common time reference for sEMG and motion signals [18].

3.2 Bluetooth Low Energy (BLE)

BLE is widely used for wearable sensing due to low power consumption and broad platform support. Sensor devices typically expose data via Generic Attribute Profile (GATT), where measurements can be delivered as notifications or indications [51]. BLE performance depends on connection parameters and radio conditions. In particular, the connection interval is a key factor that impacts latency and throughput behavior and is commonly discussed as a main tuning parameter in

BLE performance analyses [52]. For wearable streaming, packet loss and timing variability are recurring practical issues, and application layer strategies like reducing transmission frequency and bundling data have been reported to reduce loss in BLE ecosystems [53], [54].

3.3 Protocol Buffer (Protobuf)

Protobuf are a compact, language-neutral format for structured data [37], [55]. Message types are defined in a *.proto* schema and code can be generated for different languages, which supports consistent data exchange between distributed components. Compared to text-based formats, Protobuf typically reduces payload size and parsing overhead, which is useful for streaming sensor data. In practice, Protocol Buffers (Protobuf) is often combined with gRPC, where Protobuf defines the message types while gRPC provides the streaming transport (see Sek. 3.4) [39].

3.4 gRPC Remote Procedure Calls (gRPC)

gRPC is an Remote Procedure Call (RPC) framework that supports efficient streaming communication over HTTP/2 [36], [38], [39]. It is used with Protobuf to define typed messages and to keep payloads compact [37], [55]. Compared to custom link-layer solutions, gRPC offers a mature ecosystem and built-in concepts for long-lived streams and flow control [56]. However, gRPC assumes an IP-based transport and is therefore typically used between higher-level components, while wearable devices often rely on BLE links for data transfer. Independent of the transport, real-time fusion still requires explicit timestamping and buffering/alignment rules to handle asynchronous sensor streams and variable delivery timing [25], [57].

3.5 Alignment

In a combined pipeline, correct fusion depends on a consistent temporal relation between modalities [18], [50]. Because sensor streams can differ in sampling rate, buffering, and transport delay, timestamps and alignment rules are required to map samples to a common time axis [18]. Wireless links such as BLE can add

variable latency and occasional loss, which directly affects cross-stream consistency [25]–[27]. In addition, non real time operating system scheduling can introduce timing variability in embedded Linux pipelines [28]. The evaluation therefore focuses on whether the system maintains stable timing and predictable alignment under realistic operating conditions [25], [26].

Chapter 4

Implementation

This chapter explains the methods and to acquire sEMG and accelerometer signals, perform real-time fusion, and stream the results to a tablet device.

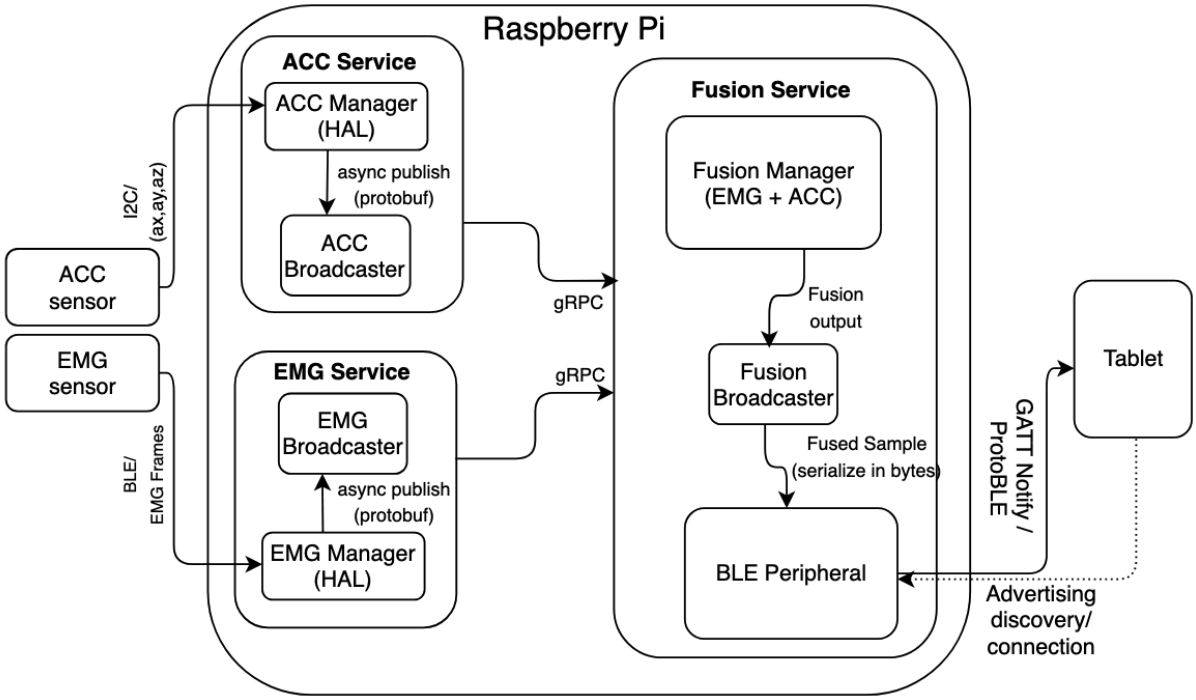


Figure 4.1: System overview.

4.1 System overview

Figure 4.1 illustrates the overall architecture of the implemented system. A Pi, which is an ARM-based Single Board Computer (SBC), serves as the central processing and communication node between the sensing hardware and the tablet

application (see Sec. 5.1). It runs Linux and provides wireless connectivity as well as standard hardware interfaces, which simplifies integration in a compact and low-power setup. The Pi receives BLE data from the wearable sensor, reads additional signals via interfaces such as I^2C timestamps the data at defined points, and forwards or fuses the streams into a unified output. The resulting stream is transmitted to the tablet for live visualization and recording. Two sensor modalities are acquired concurrently: an accelerometer connected via I^2C and an EMG sensor accessed via BLE. Both streams are captured continuously, processed on the BLE, and transformed into a single output stream that represents the system result. To keep responsibilities separated, the software on the Pi is organized into three services: an ACC service, a EMG service for EMG acquisition, and a fusion service. The accelerometer service implements the hardware access and continuously reads motion data from the sensor. The EMG service manages the wireless link to the EMG device and continuously receives EMG measurements. Each service publishes its measurements asynchronously and provides them to the fusion service through an internal interface. Communication between the services is implemented using gRPC (see Sec.2.2). Each acquisition service exposes a lightweight streaming interface that publishes sensor samples, while the fusion service acts as a client and subscribes to both streams. This approach provides a clear and typed contract between components, supports asynchronous data delivery, and allows the services to be developed and tested independently. In addition, the use of Protobuf ensures compact serialization and low overhead for high-rate streams such as the accelerometer. This modular structure allows acquisition and processing to evolve independently and simplifies testing of individual components. The fusion service subscribes to both input streams, combines information from EMG and acceleration, and produces fused output samples. These results are forwarded to the tablet through a BLE peripheral running on the Pi. For the tablet connection, the Pi advertises a custom GATT service. After the tablet connects and subscribes, fused data is transmitted as a continuous stream using GATT notifications. This design separates sensing, fusion, and wireless output while maintaining a real-time data path from the sensors to the tablet.

4.2 Hardware and runtime environment

The Pi operates in two BLE roles at the same time, the EMG service acts as a BLE client towards the EMG sensor, while the Fusion service exposes a custom GATT service as a BLE peripheral towards the tablet. The tablet discovers the device through advertising and then connects to the provided GATT interface. All services are implemented in Python and follow an event-driven runtime model based on `asyncio`. Data is exchanged via asynchronous queues, which decouple producers and consumers without blocking the event loop. Each acquisition service places new samples into a queue, while downstream components read them whenever they are ready, which naturally supports backpressure. To support multiple consumers, a broadcaster pattern is used where each subscriber receives its own queue. The broadcaster forwards each new sample to all subscriber queues, allowing fusion and logging to run independently. The queue size controls the trade-off between robustness and latency: larger queues absorb temporary load peaks, whereas smaller queues reduce latency but may drop samples when the system is overloaded. Within each acquisition service, data is produced by a hardware abstraction component and forwarded asynchronously to an internal broadcaster. The Fusion service consumes both streams through gRPC connections, performs the fusion, and forwards the resulting output to its own broadcaster. Finally, the fused data are serialized into byte payloads and transmitted to the tablet via GATT notifications (ProtoBuffer over BLE/GATT (ProtoBLE)). The GATT server functionality is implemented on top of the Linux BlueZ stack via its Desktop Bus (D-Bus) interface, while the sensor and fusion services exchange data via gRPC. The tablet acts as the user-facing client. After discovery and connection, it subscribes to notifications and receives the fused output stream continuously.

4.3 Internal transport and buffering

Acquisition, fusion, and wireless streaming run at different rates and can face short delays due to scheduling, buffering, or the wireless link. To prevent these delays from affecting downstream stages, the system uses a lightweight publish–subscribe layer based on the *Broadcaster* abstraction [58]. A Broadcaster acts as a data channel between producers and consumers. Producers publish samples, and each consumer subscribes with its own buffered queue, as shown in Figure 4.2. This decouples acquisition from processing and can help to prevent a slow consumer

from blocking the rest of the system. Separate channels are used for fused output, which is then streamed to the tablet. Each subscriber queue has a fixed maximum size to keep latency bounded. If a queue becomes full, the broadcaster applies an overflow rule and drops samples instead of blocking. Each subscriber uses a First In First Out (FIFO) queue, and in the overflow case the oldest queued sample is discarded to make room for the newest one. The oldest item is removed and the newest sample is kept, prioritizing fresh data for real-time monitoring (*drop-oldest*) [59].

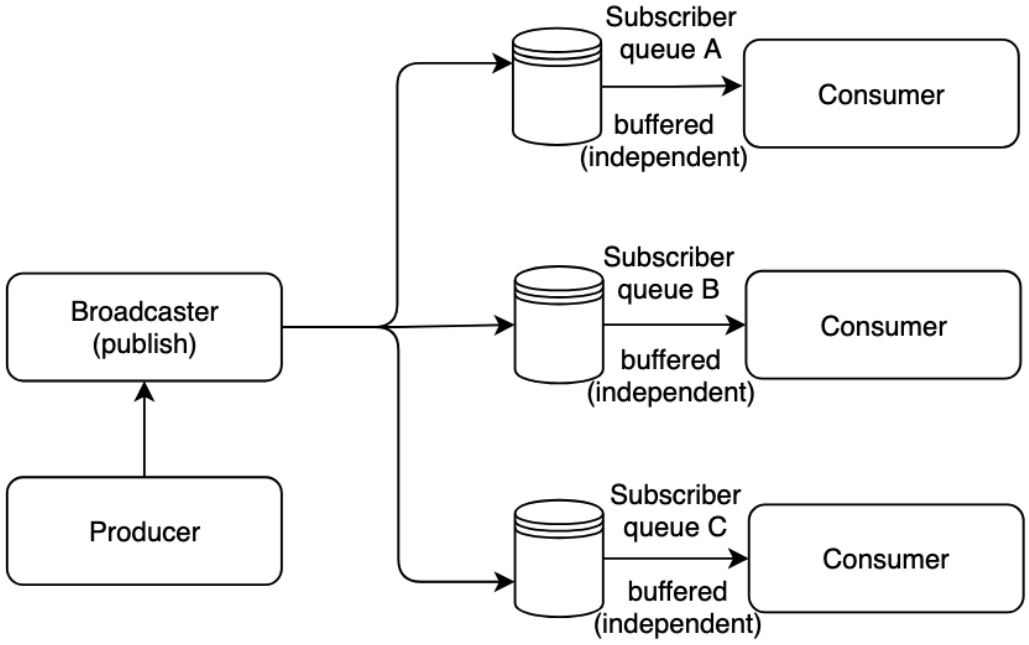


Figure 4.2: Broadcaster-based internal transport with per-subscriber queues.

4.4 Sensor acquisition

The system acquires accelerometer and EMG data continuously on the Pi. Since both modalities were introduced in the theoretical background (2.1.2, 2.1.1), this section explains the acquisition path in Fig. 4.1 and how both signals are provided to the fusion pipeline. Two design principles guide the implementation: timestamps are generated close to where data is read or received to reflect the actual arrival time on the host and to avoid errors from later buffering or processing, and acquisition is decoupled from fusion so that delays in later stages do not slow

down sensor sampling.

4.4.1 ACC Manager (HAL)

The ACC Manager implements the hardware abstraction for the inertial sensor and reads acceleration through the I^2C interface. It executes a periodic acquisition loop, obtains the three axes (a_x, a_y, a_z), converts the raw readings into physical units, and assigns a local timestamp to each sample. The measurements are then published asynchronously to the ACC Broadcaster's queue. The Fusion service accesses the broadcaster's sample queue via a gRPC streaming connection.

4.4.2 EMG Manager (HAL)

The EMG Manager implements the hardware abstraction for EMG acquisition and receives EMG data from an external device over BLE. After establishing a connection as a BLE client, it enables streaming by subscribing to notifications of the relevant GATT characteristic. Incoming payloads are timestamped at reception, and published asynchronously to the EMG Broadcaster. The Fusion service subscribes to this EMG stream via an internal gRPC connection to the EMG service. To support continuous operation, the EMG Manager monitors the connection state and can recover from temporary link interruptions. In contrast to the accelerometer loop, the EMG acquisition is event-driven: samples are processed on arrival of BLE notifications, so the effective update period is determined by the sensor and the BLE link rather than by a host-side timer.

4.5 Protocol Buffers in HAL interface

Figure 4.1 shows how the HALs separate hardware access from fusion processing. The ACC Manager and the EMG Manager read the raw sensor data and convert them into time-stamped Protobuf messages. These messages are published internally via broadcasters (see Sec. 4.3). The ACC and EMG services expose these Protobuf messages as continuous streaming sequences, so that samples are transmitted incrementally as they arrive. The Fusion Manager subscribes to the EMG and ACC streams as ordered sequences. After fusion, the output is again a Protobuf message. It is serialized into bytes and sent to the tablet via BLE GATT notifications (ProtoBLE) after the connection is established. This keeps

the message format consistent, even if the transport changes later.

4.6 Fusion manager

This work applies two fusion modes to assess the impact of embedded reprocessing. The feature-based mode follows the sliding window with fixed-step timing model from Section 2.3.5. In contrast, the forwarder mode operates without windowing and directly maps incoming sensor samples to the fused output.

4.6.1 Alignment

The EMG and ACC streams have different sampling rates and may not arrive with a jitter due to buffering or wireless disturbances. For fusion, both modalities must be mapped to a consistent time reference. In this work, the target output time base is 1 kHz, so the fusion output follows a uniform timing even if the input streams are irregular. Alignment is handled differently in the two fusion modes: In the feature-based mode, alignment follows the sliding-window model with a discrete fusion clock (see Sec. 2.3.5). At each tick t_k, the system forms a window W_k and computes one output on a 1 kHz time base. If a modality provides too few samples in W_k, the most recent valid value is reused (see Sec. 2.3.4). If a modality provides more samples than required, the stream can be limited by rate throttling (see Sec. 2.3.3) (see Fig. 4.3). In the forwarder mode, there is no fusion clock and no windowing. Instead, the system publishes one fused output whenever a new sample arrives from EMG or ACC. The output timestamp equals the timestamp of the triggering sample. The other modality contributes its most recent stored value using sample-and-hold (see Sec. 2.3.4). To avoid pairing samples that are too far apart in time, an optional freshness limit (`MAX_DELAY`) can be applied. If the held sample is older than the allowed delay, it is marked invalid and replaced by a neutral value (zeros).

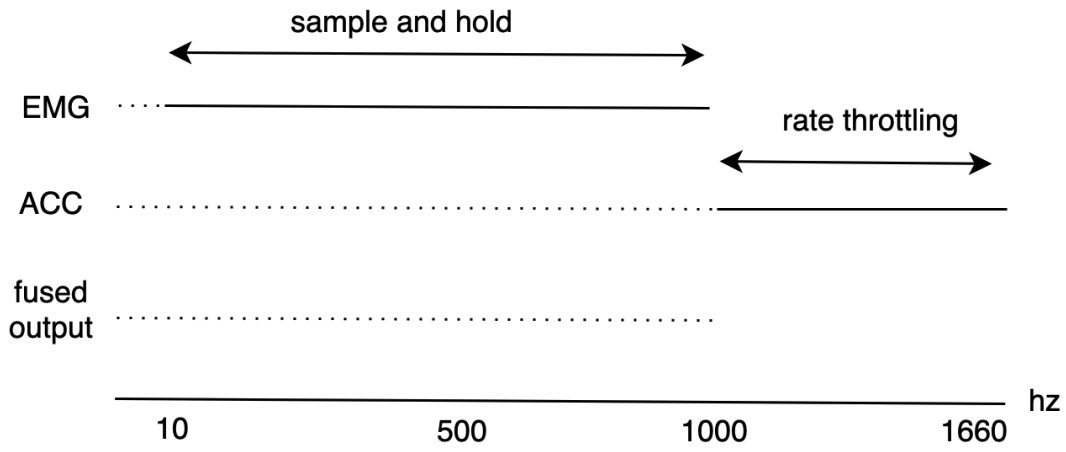


Figure 4.3: Samples alignment

4.6.2 Forwarder fusion

In the forwarder fusion, the system forwards values with minimal processing. The fusion publishes one fused output whenever a new sample arrives from EMG or ACC. The fusion stage stores the latest sample of each modality. When one modality delivers a new sample, this sample triggers the output and defines the output timestamp. The system applies uses the most recent stored value of the other modality to the fused sample (see Sec. 2.3.4). To reduce misaligned pairings, a modality-specific maximum hold time (`MAX_DELAY`) is applied. A held sample is considered valid only if its timestamp is within `MAX_DELAY` of the trigger timestamp. Otherwise, the held sample is treated as stale: it is marked as invalid and replaced by a neutral value (zeros). This maintains a continuous output stream while preventing the reuse of outdated sensor data. In our configuration, `MAX_DELAY` differs between modalities because the streams have very different update rates. For the high-rate ACC stream, `MAX_DELAY` is set to 100 ms to avoid reusing motion samples that are no longer representative of the current state. For the low-rate EMG stream, `MAX_DELAY` is set to 4000 ms so that the most recent EMG value remains available between low-frequency updates and short transmission gaps.

4.6.3 Feature-based fusion

In the feature-based fusion mode, each window W_k is mapped to a compact feature vector, consistent with the real-time feature definitions in Chapter 2. For the accelerometer, the scalar motion level is computed from the acceleration magnitude Eq. (2.1). In addition, short-term variability is summarized by the variance of the magnitude within the window Eq. (2.3). For EMG, the activation level is computed as the windowed RMS Eq. (2.2). The fused output therefore consists of one feature vector per window and follows the mapping $f[k] = g(W_k)$ introduced in Section 2.3.5. The window length is set to 200 ms. A fixed step size of $\Delta = 1$ ms is used to achieve a 1 kHz output rate. This means that every 1 ms a new feature vector is produced from the most recent 200 ms of sensor data via the function $g(W_k)$ (see Sec. 2.3.5) and published as the fused output.

4.6.4 Fused output stream

The fusion module generates a continuous fused output stream that combines EMG and ACC. A fused sample is created either at feature-based fusion instants or forwarder fusion when a new sensor sample arrives. In both cases, each output sample is assigned a timestamp and represents the best available pairing of both sensor modalities at that moment. The fused stream is then passed to the output interface and forwarded to the tablet.

4.7 BLE Peripheral

The BLE Peripheral provides the wireless interface to the tablet and represents the final stage of the data path in Fig. 4.1. It makes the Pi discoverable by advertising a custom GATT service. After the tablet connects, two interactions are supported: the tablet sends configuration or subscription settings via GATT write requests, and it receives the fused output as a continuous stream via GATT notifications. In this implementation, the tablet link follows a ProtoBLE approach. Both control messages and fused samples are represented as Protobuf messages, serialized into compact byte payloads, and transmitted through the corresponding GATT characteristics. The peripheral manages the connection state so that streaming starts only when a client is connected and stops cleanly if the link is lost. This keeps the fusion logic independent of wireless transport details and allows the same streaming concept to be realized with an IP-based alternative

such as gRPC if required. Overall, the interface follows the server-side streaming idea introduced in Sec. 2.2: a client issues a single subscription/configuration request and then receives an ongoing stream of messages, while the underlying transport is BLE/GATT instead of HTTP/2.

4.8 Flutter Application

A *Flutter*-based mobile application was implemented as a lightweight client to monitor the system during experiments. The app connects to the embedded device via BLE and subscribes to a notification stream. After the connection is established, the client can send a small configuration message and then receives fused samples continuously as binary payloads. On the smartphone, these payloads are decoded and displayed in real time using live charts and simple status indicators. In addition, reconnect handling is included so that short connection interruptions can be recovered without restarting the measurement setup. *Flutter* supports cross-platform development (e.g., Android and iOS), which allows using a single code base instead of implementing a separate application for each platform. In the project the app was expose for Android Tablet.

4.9 Timing measurements

To evaluate the timing behavior of the streaming pipeline, timestamps are recorded at selected measurement points along the processing chain as shown in Fig. 4.4. Only for the intern sides Pi Time behavior were relevant and for *Flutter-based* application were not considerd. Following the definitions in Sec. 2.3.7, we log timestamps at the HAL input ($t_i^{\mathrm{HAL,in}}$), at the HAL output ($t_i^{\mathrm{HAL,out}}$), at the fusion input ($t_i^{\mathrm{F,in}}$), and at the fusion output ($t_i^{\mathrm{F,out}}$). All timestamps are stored as epoch timestamps (absolute time since the Unix epoch) with microsecond resolution. The index i is used to match timestamps that belong to the same logical item. These measurement points form the basis for computing delay, jitter, and bandwidth in the experimental setup.

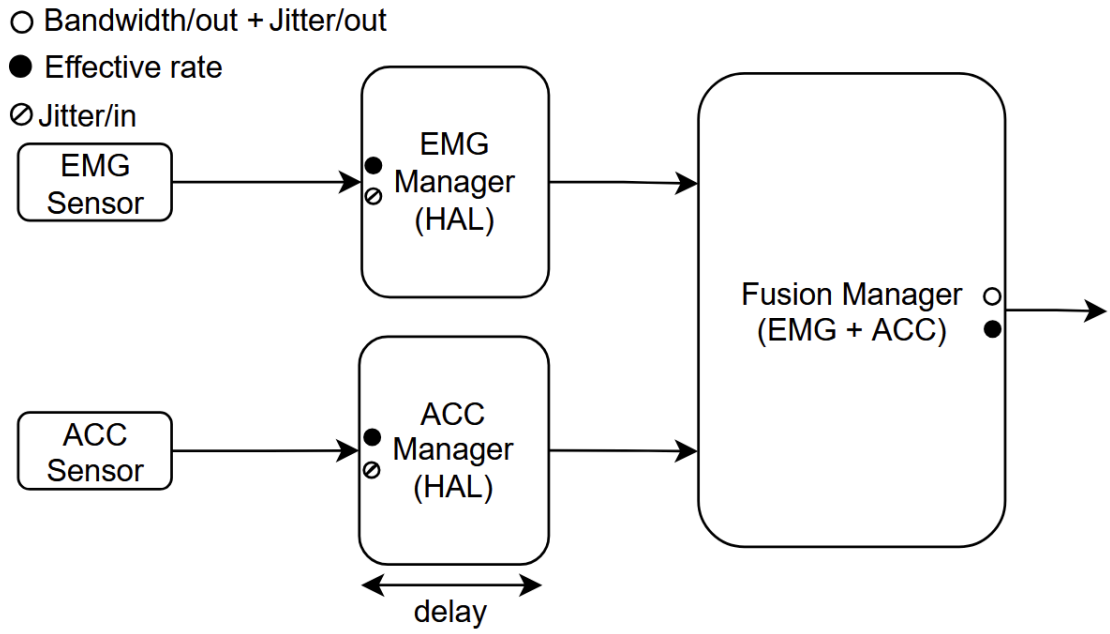


Figure 4.4: Measurement points used to evaluate delay, jitter, and bandwidth in the streaming fusion pipeline.

4.10 Data analysis

During the experiments, timing events are written to log files. Each entry stores a timestamp in µs, the measurement point (HAL input, HAL output, fusion input, fusion output), and an item identifier to match events that belong to the same logical sample. After each run, the logs are processed offline with a Python script. The script parses the files, pairs timestamps by item identifier, and computes effective rate, delay, jitter and bandwidth between measurement points as defined in Sec. 2.3.7.

Chapter 5

Experimental Setup

This chapter describes the experimental tests, the used hardware and software, and the main configuration parameters.

5.1 Setup

For the experiments, two Pi boards are used: a Pi Zero 2 W and a Pi 5. As sensors, we used an LSM6DSOX accelerometer (see Sec. 2.1.2) and an ANR M40 sEMG sensor (see Sec. 2.1.1). A Samsung tablet was used as the client device for receiving and visualizing the streamed data. A power bank supplied the Pi during the tests. The accelerometer was connected to the Pi via the I^2C interface. Two pull-up resistors were added on the bus lines to ensure a stable connection, and the resistors and wires were soldered together. The EMG sensor was placed on the arm (flexor carpi radialis) to record muscle activity. To simulate a wearable patient device, the was attached to the same arm using an elastic armband as showing in Figure 5.1. This way, the system could be tested under realistic movement conditions.

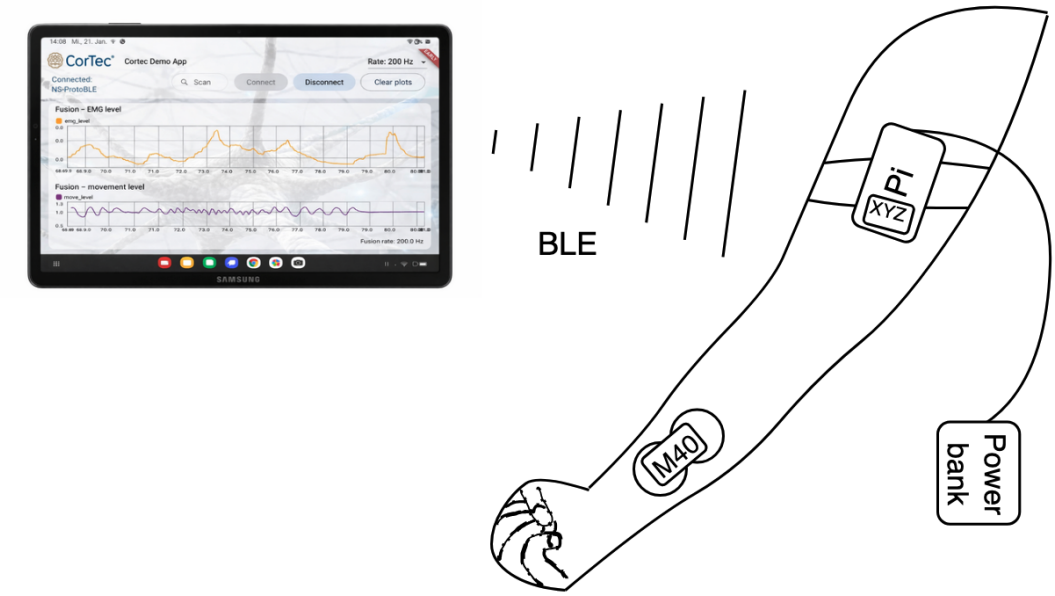


Figure 5.1: The setup.

5.2 Configuration

All experiments were run with fixed acquisition, fusion, and transmission settings to ensure comparability across test runs. The ACC LSM6DSOX was configured to stream at 833 Hz and connected via the I^2C bus (see Sec. 2.1.2)[35]. The EMG sensor (ANR M40) was operated with its factory settings and provided an update rate of up to 10 Hz [33]. To avoid unrealistic peaks, EMG amplitudes were bounded in software.

5.3 Software and services

On the Pi, Pi OS Lite was installed on a 64 GB microSD card. All software components were started as a single system service separated. This services launched the ACC manager (HAL), the BLE manager (HAL), and the fusion manager. BLE advertising and the GATT server were also handled in the same service as fusion manager. All timing measurements (see Sec. 4.9) were performed under this configuration, and the recorded data were evaluated as described in Sec. 4.10.

5.4 Software libraries

The software environment and the main libraries are listed with their versions. The Pi pipeline runs on Python 3.11 and uses modules from the Python standard library (e.g., asyncio, time, struct). Since these modules are part of Python, they are not listed individually.

Table 5.1: Pi side.

Name	**Version**	**Used for**
Python	3.11	Runtime environment
Bleak	0.22.3	BLE communication
dbus_next	0.2.3	BlueZ D-Bus interface
grpcio	1.67.0	gRPC streaming between components
smbus2	0.4.3	I^2C sensor readout
protobuf	5.28.2	Message serialization

Table 5.2: Flutter side.

Name	**Version**	**Used for**
Flutter	3.24	App framework
flutter_reactive_ble	3.5	BLE client and notifications
fl_chart	0.68.0	Live plotting
permission_handler	11.3.0	Runtime permissions
wakelock_plus	1.2.5	Preventing sleep during recording
fixnum	1.1.0	Fixed-width integers (protobuf support)
Dart protobuf / grpc	3.1.0 / 4.0.0	Protobuf messages and gRPC support

Table 5.3: Libraries used for data evaluation.

Name	Version	Used for
Python	3.12.1	Evaluation runtime
pandas	2.3.3	CSV parsing and aggregation
NumPy	1.26.0	Numeric processing
Matplotlib	3.10.7	Plots and figures

5.5 Tablet Application

A Samsung tablet was used as the client device. It received the fused data from the Pi over BLE. On the tablet, a Flutter/Dart app was also developed (see Sec. 4.8). The app connects to the Pi, subscribes to BLE notifications, and decodes the received messages into fusion samples. It shows the fused output as a live plot, so the signal can be checked during the experiment. The app also provides basic controls, such as connect/disconnect and start/stop streaming. If needed, the received data can be stored on the tablet for later analysis. During the experiments, the Pi transmitted fused samples to the app at 200 Hz (see Fig. 6.1).

5.6 Measurement Session

The measurements were run once and lasted 5 min. During the session, the streamed data were recorded and stored as CSV files. The recorded datasets were evaluated locally as described in Sec. 4.10.

Chapter 6

Results and Discussion

This chapter reports the timing measurements of the multisensor acquisition and fusion pipeline described in Chapter 4 and evaluated under the fixed configuration of Chapter 5. The goal is to quantify real-time performance from sensor reception on the Pi to the fused output received by the tablet. We analyze jitter of inter-arrival times, effective output rate, sample loss, and application-layer bandwidth based on the timestamp markers defined along the processing chain. The results are discussed with respect to the research question of whether the system can sustain an internal fusion time base of 1 kHz while maintaining stable wireless streaming with acceptable latency and variability. In addition, the two fusion modes (*feature-based* and *forwarder*) are shown to highlight how windowing, sample-and-hold, buffering, and BLE/GATT behavior contribute to the observed timing characteristics.

6.1 Live visualization on the tablet

Figure 6.1 shows the tablet application during a measurement. The app receives the fused output stream from the Pi via BLE and displays the two main signals in real time. During the experiments, the stream to the tablet was configured to 200 Hz which is enough for live plotting. This live view was mainly used to verify continuous streaming and to detect connection issues during recording.



Figure 6.1: Flutter/Dart Application (*Feature-based fusion*).

6.2 Hardware feasibility

After about one minute of runtime with Pi Zero 2 W, the BLE connection became unstable and finally disconnected. Reconnect attempts often failed until the Bluetooth subsystem was restarted. Overall, the Pi Zero 2 W could not handle the required two connections at the same time: it had to connect to the EMG sensor and also keep a stable link to the tablet for streaming. With both links active, the system became unstable and eventually crashed, so this platform was not suitable for reliable and repeatable measurements. In contrast, the Pi 5 maintained stable connections to both devices over the full test duration. For this reason, the Pi 5 was used as the main hardware platform for the remainder of the project.

6.3 Effective rate

Effective rate was evaluated as described in Sec. 2.3.7

6.3.1 Accelerometer

The accelerometer was configured for a nominal rate of 833 Hz (see Sec. 2.1.2). As shown in Fig. 6.2, the measured effective rate (see Sec. 2.3.7) at the HAL entry was clearly below this target. Over the full recording, the mean effective rate was 650.8 Hz and the median was 647 Hz, corresponding to a reduction of 21.9 % relative to the nominal 833 Hz. After a short start-up phase, the rate remained largely stable, about 50 % of the values lie within 640-660 Hz, with only sometimes higher outliers approaching 700 Hz. The reduced rate is mainly caused by host-side readout overhead, in particular the blocking I^2C register reads required to fetch each sample, which introduce per sample latency and limit the achievable throughput. In addition, acquisition and publishing are implemented as two asynchronous tasks separated by a queue. To avoid blocking when one stage temporarily lags behind, the synchronization uses a small artificial wait time. While this prevents the system from freezing, it adds scheduling overhead and can delay queue draining, which further lowers the effective rate under load.

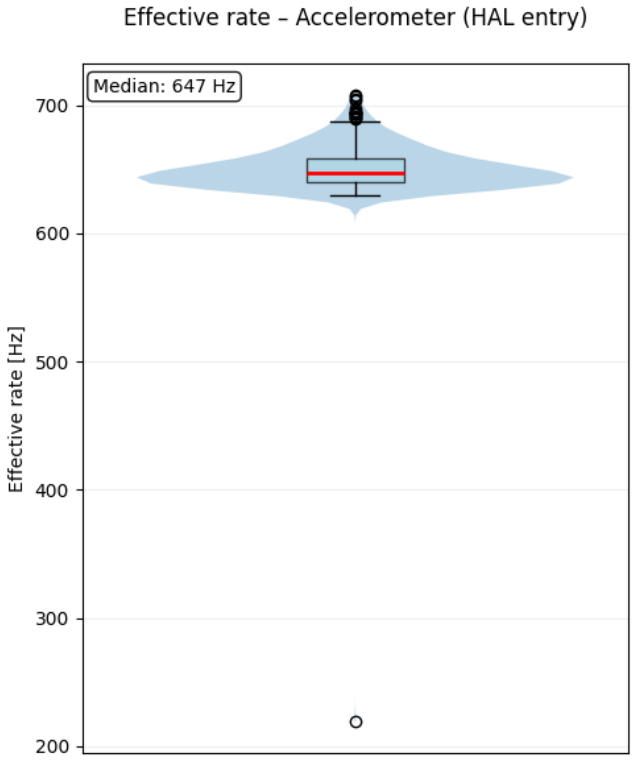


Figure 6.2: Effective Accelerometer rate at HAL entry.

6.3.2 EMG

The EMG sensor was used with factory settings and a nominal sampling rate of 10 Hz (Sec. 2.1.1). The sensor always batches two consecutive samples into one BLE notification, i.e., it sends one notification about every 200 ms instead of one sample every 100 ms. As result, the rate observed at the HAL entry based on notification arrivals is 5.0 Hz (Fig. 6.3). Although each notification contains two consecutive samples, the current HAL implementation decodes both samples, therefore, the effective sampling rate in the pipeline is limited to 5 Hz, rather than the nominal 10 Hz. Across the full recording, the effective rate remains close to 5.0 Hz (median: 5.0 Hz, mean: 5.0 Hz), with only rare outliers around 4-6 Hz.

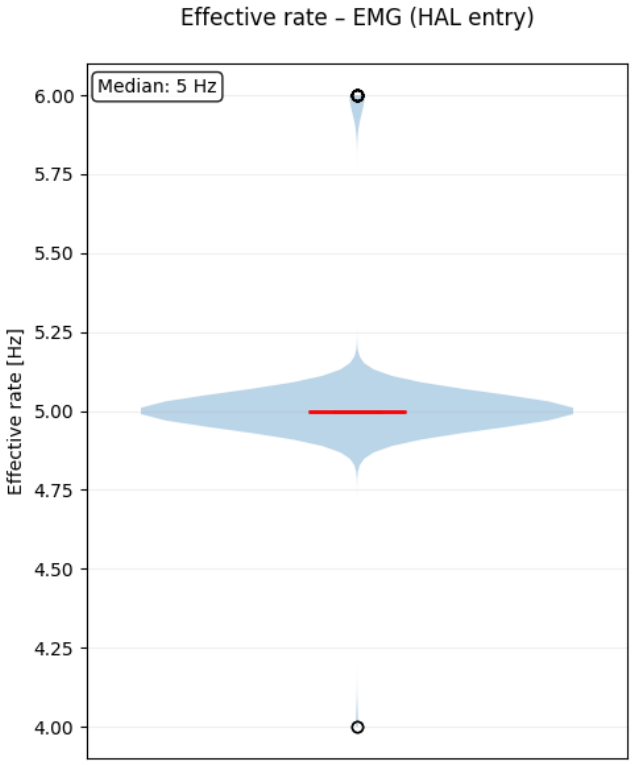


Figure 6.3: Effective EMG rate at HAL entry (two samples per notification).

6.3.3 Forwarder-based fusion

To assess the best-case performance of the *forwarder* pipeline, we use the effective rates already available at the HAL entry as the upper bound for forwarding, namely 647 Hz for the accelerometer and 5 Hz for EMG. In the forwarder-based fusion, the fused output is mainly driven by the accelerometer stream, as it updates much faster than the EMG stream. Therefore, in the best case the expected fusion output rate is close to the accelerometer rate observed at the HAL, i.e., about 648 Hz. As shown in Fig. 6.4, the measured fusion output rate is close to this limit, with a mean of 659.3 Hz and a median of 656 Hz. Since this rate is close to the accelerometer rate already observed at the HAL entry (see Sec. 6.3.1), the *forwarder* and fusion stages themselves add no further delay. Thus, the main cause of the reduced output rate is upstream in the host-side acquisition and transport path, not the forwarder-based fusion.

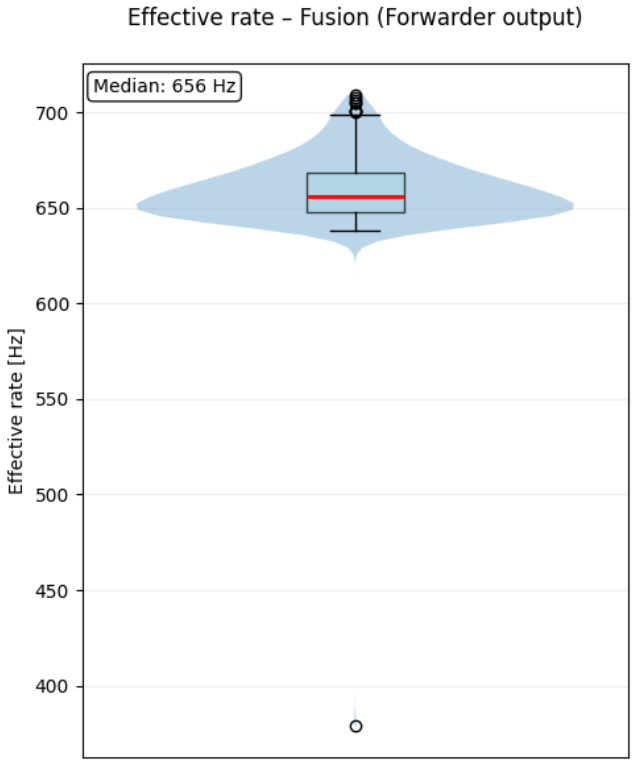


Figure 6.4: Effective rate at the forwarder-based fusion output.

6.3.4 Feature-based fusion

To check the performance of the *feature-based* fusion algorithm, its effective output rate was measured from the sensor acquisition path (see Sec. 6.3.1 and 6.3.2). In contrast to the *forwarder-based* fusion, where sensor streams are forwarded individually, the *feature-based* approach produces a single fused output stream. As shown in Fig. 6.5, the algorithm generated a stable output rate of 1000 Hz. Over the full recording, the mean effective rate was 1000 Hz, which matches the nominal value. Only rare outliers around 350 Hz were measured. This indicates that the feature algorithm can run at 1000 Hz and is therefore not the main reason for a lower end-to-end system rate.

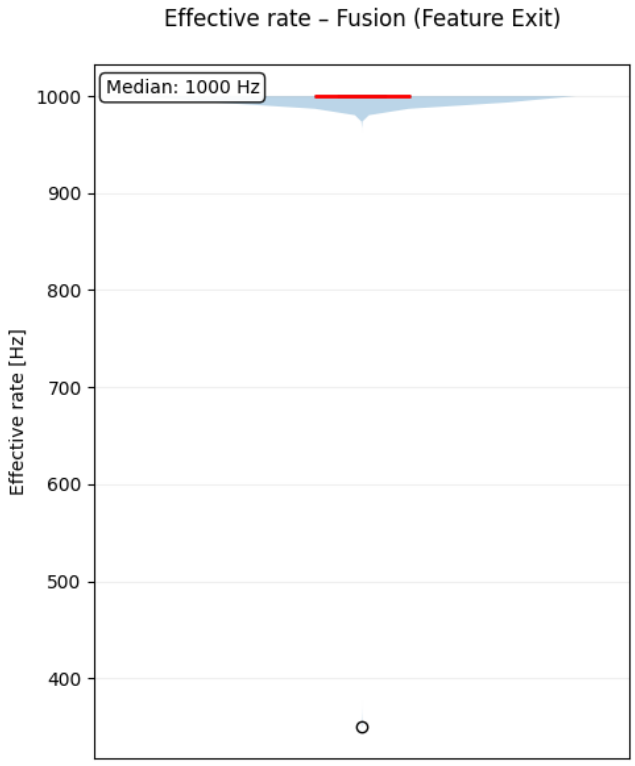


Figure 6.5: Effective rate at the feature-based fusion output.

6.4 Jitter, delay and bandwidth

Jitter, delay and bandwidth were evaluated as described in Sec. 2.3.7.

6.4.1 Jitter and Effective Period

Figure 6.6 shows the measured periods at the accelerometer HAL entry. The accelerometer was configured for 833 Hz (see Sec. 5.2), corresponding to a nominal period of 1.20 ms. The median period is 1.147 ms. The distribution has two clear clusters (1.2 ms, 2.3 ms), which means that two timing regimes occur repeatedly during the recording. The second cluster around 2.3 ms is close to twice the nominal period and suggests that, in these intervals, one sample is effectively not observed at the HAL entry i.e., only every second value is forwarded or logged. A likely reason is host-side behaviour (e.g., buffering and task scheduling). Depending on the current system load, samples can arrive either close to the nominal pacing or in short bursts. This produces two groups of inter-arrival deviations. Figure 6.7 shows the measured periods at the EMG HAL entry. The median is about 197 ms, which is much larger than for the accelerometer. This matches the lower update rate and the less regular timing of the BLE-based stream. In addition, some of the points were at 0 ms. This can be explained by batching: the sensor transmits two sequential samples within one BLE notification. When such a notification arrives, the two samples can be immediately handled, followed by a longer waiting time until the next notification. Finally, Fig. 6.8 shows the period at the *forwarder-based* fusion output. The median is 1.126 ms, which is very close to the accelerometer value. This indicates that the *forwarder-based* fusion output timing is mainly driven by the high-rate ACC stream. Since the fusion publishes an output whenever a new ACC or EMG sample arrives, the more frequent ACC updates dominate the trigger events. Therefore, the *forwarder-based* fusion stage does not introduce its own output schedule, but mostly forwards the timing characteristics of the trigger stream. Finally, Fig. 6.9 shows the jitter at the fusion feature output. The inter-output interval is almost constant at 1 ms (median 1 ms). This is expected because the feature fusion runs on a fixed 1 ms time base: each fusion step advances by exactly 1 ms, and the output timestamp is generated on this schedule. As a result, the output timing is mainly defined by the internal fusion clock rather than by the irregular arrival patterns of the input streams.

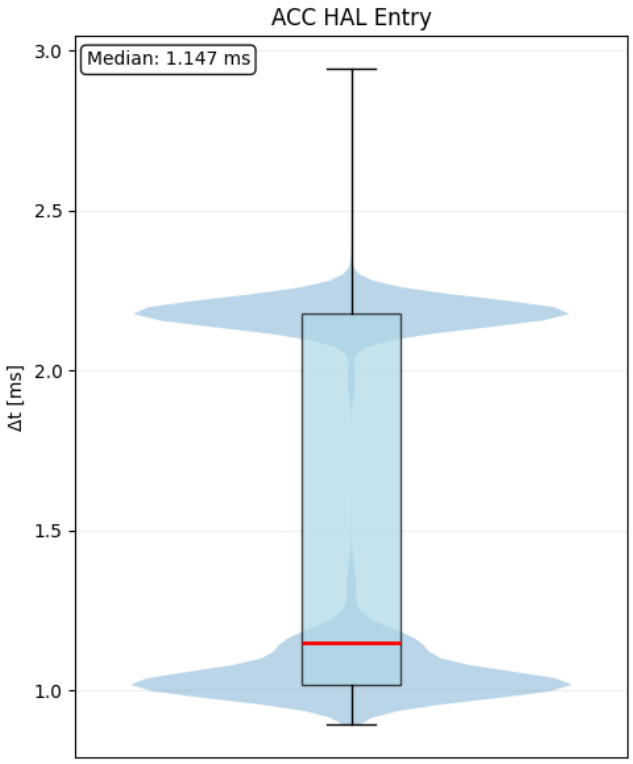


Figure 6.6: Jitter at the accelerometer HAL entry.

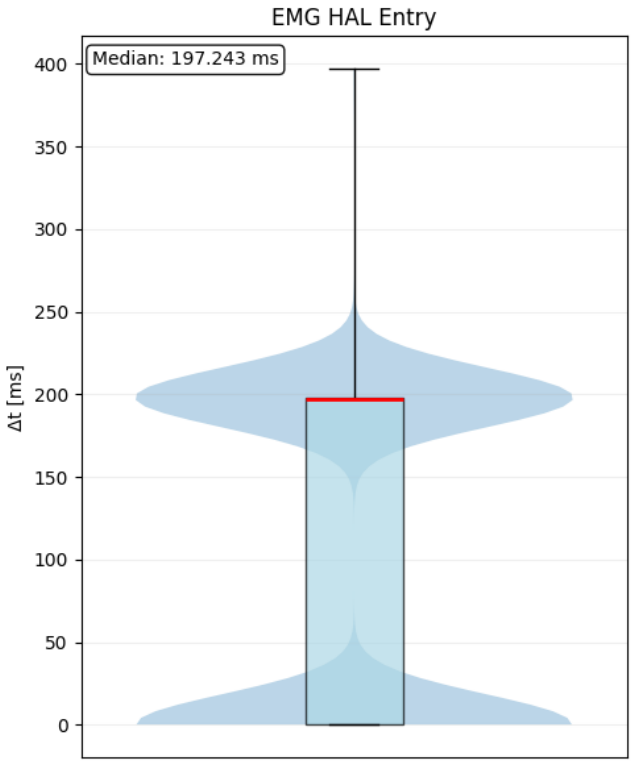


Figure 6.7: Jitter at the EMG HAL entry.

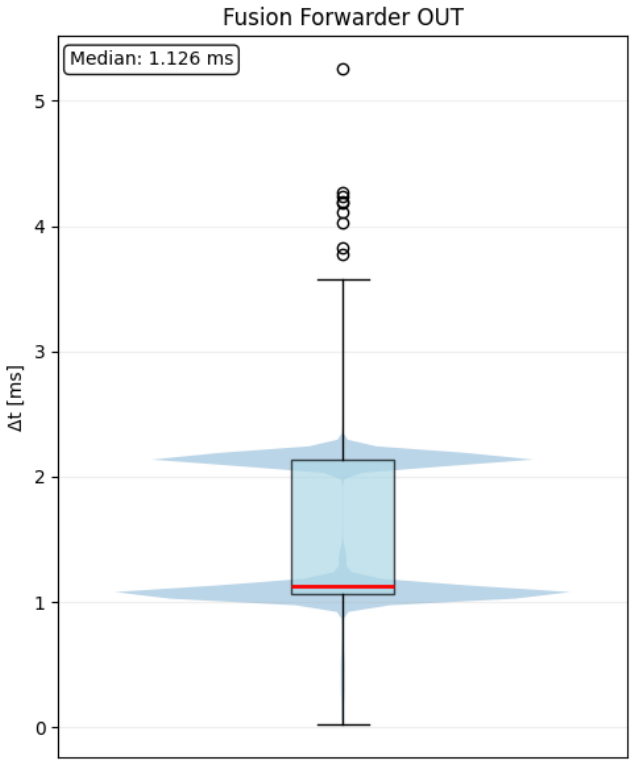


Figure 6.8: Jitter at the Fusion Forwarder output.

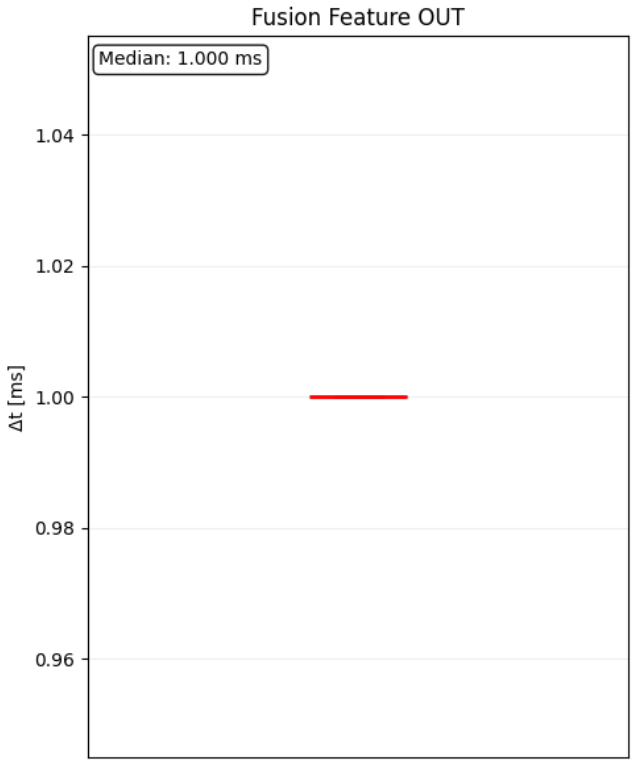


Figure 6.9: Jitter at the Fusion feature output.

6.4.2 Delay

Figure 6.10 summarizes the HAL processing delay for the accelerometer from entry to output. The median delay is 0.026 ms. Figure 6.11 shows the matching EMG HAL delay, with a median of 0.137 ms. While a few outliers are visible, the typical delays stay well below 1 ms. This indicates that both HAL paths mainly perform lightweight processing and forwarding. Overall, these results suggest that the main timing variability is created before the HAL. For EMG, wireless transmission and BLE notification timing are plausible sources. For ACC, host-side buffering and scheduling can explain the observed variation at the entry point. In contrast, the HAL itself adds only a small and mostly stable contribution to the overall timing behaviour.

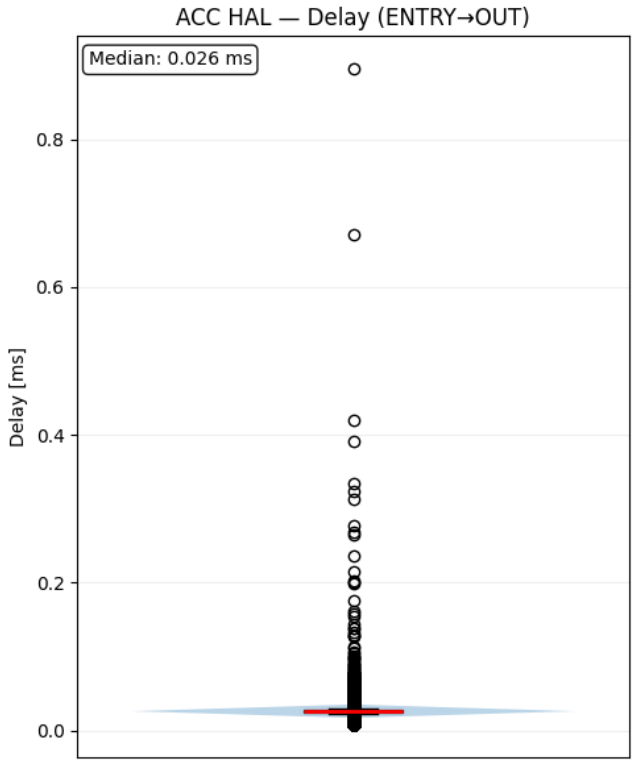


Figure 6.10: Delay within the accelerometer HAL measured from `ENTRY` to `OUT`.

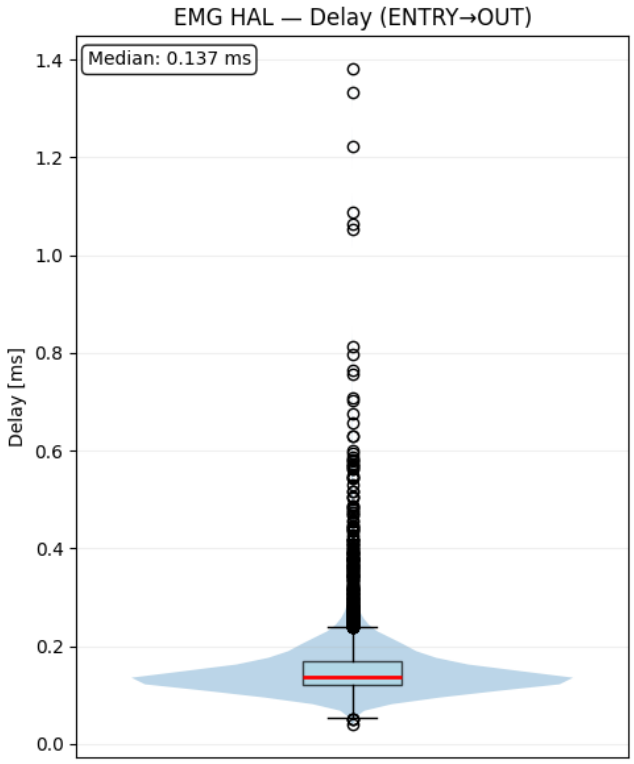


Figure 6.11: Delay within the EMG HAL measured from `ENTRY` to `OUT`.

6.4.3 Bandwidth

The average payload bandwidth at the final fusion output was 168.9 kbit/s for the forwarder algorithm and 256 kbit/s for the feature algorithm (see Sec. 2.3.7). Based on the output rates (650 Hz forwarder, 1 kHz feature) and an average payload size of about 32 B per sample, the expected payload bandwidth is approximately 166.4 kbit/s and 256 kbit/s, respectively. Compared to the forwarder pipeline, the feature pipeline therefore requires 51.5 % higher application-layer payload bandwidth in this configuration.

Chapter 7

Conclusion

This thesis investigated the timing behavior of a multisensor pipeline on a Pi that combines a BLE-based EMG stream and a high-rate ACC stream. The main goal was to evaluate whether the system can sustain an internal fusion time base of 1 kHz with stable timing. The measurements show that the overall timing is mainly limited by the acquisition paths. The ACC was configured for 833 Hz, but the effective rate at the HAL entry was clearly lower (median about 647 Hz). This is mainly caused by host-side overhead, especially the blocking per-sample reads over I^2C, which add latency for every sample and therefore limit the achievable throughput. Additionally, the acquisition and publishing stages run as separate asynchronous tasks with a queue in between, and the small synchronization wait time introduces extra scheduling overhead under load. The EMG stream shows irregular arrivals due to the BLE transmission behavior and batching/burst. In addition, the current HAL implementation processes all samples contained in each notification, which helps maintain the effective EMG rate within the pipeline. Two fusion modes were evaluated. In the forwarder mode, fusion is event-driven and publishes one output whenever a new EMG or ACC sample arrives. Therefore, the relevant reference is the effective input rate at the HAL entry, because the forwarder can only publish when samples are actually available in the pipeline. In the measurements, the forwarder output rate is close to this available input rate and can be slightly higher than the ACC rate at the HAL entry because both ACC and EMG can trigger an output, so EMG events add a small number of additional publications. For the ACC path, the effective rate at the HAL entry is noticeably lower than the configured sensor rate. With a configured 833 Hz and a measured median of about 647 Hz, the deviation is about 22 %. This deviation is mainly created upstream by acquisition and transport, not by the forwarder logic. The same principle also applies if a sensor is configured for 1 kHz: the forwarder can

only publish at the rate that actually arrives at the HAL entry, so any upstream loss of effective rate will appear as a similar relative deviation at the forwarder output. In the feature-based mode, fusion is computed using a fixed-step sliding-window approach with $\Delta = 1\,\text{ms}$. This defines a 1 kHz fusion time base. The effective output rate of the feature-based fusion was measured separately from the sensor acquisition path. The same sensor configuration was used (ACC 833 Hz). However, at the HAL entry the recorded rate showed a deviation of about 22 %. Although the input rate was reduced and irregular, the algorithm produced a stable fusion output rate of 1 kHz over the full recording (mean 1 kHz). This indicates that the feature-based fusion can maintain an internal 1 kHz time base and that the fusion computation is not the limiting factor for the end-to-end rate. Overall, the feature-based mode maintains a 1 kHz fusion time base at the output, but both acquisition paths already show about a 22 % sample loss at the HAL entry. Therefore, the end-to-end performance of both modes is primarily limited by the effective input rate and timing irregularities at the HAL entry rather than by the fusion stage itself.

7.1 Outlook

Future work should focus on improving the acquisition paths, because they dominate the timing behavior of the overall system. For the ACC path, enabling and using the sensor register FIFO is a promising next step. Reading blocks of samples instead of individual samples can reduce the per-sample I^2C overhead and should increase the effective rate at the HAL entry while reducing jitter. For the EMG path, the current sensor is not ideal for strict timing requirements, because the BLE stream is irregular and the effective rate is limited by batching. If this EMG sensor must be used, the BLE link should be tuned carefully, because the update behavior depends on both firmware and connection settings. Relevant parameters include the Physical Layer (PHY), the connection interval, the connection event length (connection window), slave latency, and the maximum GATT Maximum Transmission Unit (MTU) size (and related data length settings). With optimized settings, it may be possible to reduce batching and move closer to the nominal behavior. As an alternative, a BLE-based EMG sensor such as the *muscleBAN* device is more suitable for the target of high-rate real-time processing, because it can provide 1 kHz EMG data and additionally includes integrated motion sensing (ACC and a gyroscope) at 1 kHz [60]. With such a sensor, the forwarder mode could also reach approximately 1 kHz at the output if the HAL entry receives clean 1 kHz updates

Chapter 8

Acknowledgments

I would like to thank Prof. Dr. Thomas Stieglitz for giving me the opportunity to write this thesis and for supporting my research. I am grateful to CorTec GmbH for enabling this work and for funding my research. I also want to thank all my colleagues at CorTec for their support, guidance, and helpful discussions. I would like to thank Christian Stolle for welcoming me into his department and for giving me the chance to carry out this thesis there. Special thanks go to Dr. Michael Burdumy for his help with debugging and for carefully proofreading this thesis. I also thank Feras Shaheen for his support with the project architecture and for additional help during debugging. Furthermore, I would like to thank Lukas Huth for his assistance in optimizing the BLE connection. Finally, I would like to thank Dr. Martin Schüttler for his support.

Bibliography

[1] Google, *Notebooklm: Ai research tool & thinking partner*, https://notebooklm.google/, Online; accessed 2026-02-17.

[2] DeepL SE, *Deepl translator*, https://www.deepl.com/en/translator, Online; accessed 2026-02-17.

[3] DeepL SE, *Deepl write: Ai-powered writing companion*, https://www.deepl.com/en/write, Online; accessed 2026-02-17.

[4] OpenAI, *Chatgpt: Ai chatbot to discover, learn & create*, https://openai.com/chatgpt/overview/, Online; accessed 2026-02-17.

[5] GitHub, Inc., *Github copilot: Your ai pair programmer*, https://github.com/features/copilot, Online; accessed 2026-02-17.

[6] GitHub, Inc., *Github copilot documentation*, https://docs.github.com/copilot, Online; accessed 2026-02-17.

[7] draw.io Ltd, *About draw.io / diagrams.net*, https://www.diagrams.net/about, Online; accessed 2026-02-17.

[8] P. Raghavan, "Upper limb motor impairment after stroke," *Physical Medicine and Rehabilitation Clinics of North America*, vol. 26, no. 4, pp. 599–610, Nov. 2015, Accessed: 2026-01-26. DOI: 10.1016/j.pmr.2015.06.008. [Online]. Available: https://doi.org/10.1016/j.pmr.2015.06.008.

[9] C. J. Winstein, J. Stein, R. Arena, *et al.*, "Guidelines for adult stroke rehabilitation and recovery," *Stroke*, vol. 47, no. 6, e98–e169, Jun. 2016, Accessed: 2026-01-26. DOI: 10.1161/STR.0000000000000098. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/27145936/.

[10] L. A. Ingram, A. A. Butler, M. A. Brodie, S. R. Lord, and S. C. Gandevia, "Quantifying upper limb motor impairment in chronic stroke: A physiological profiling approach," *Journal of Applied Physiology (1985)*, vol. 131, no. 3, pp. 949–965, Sep. 2021, Accessed: 2026-01-26. DOI: 10.1152/japplphysiol.

00078.2021. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/34264125/.

[11] C. Marquez-Chin and M. R. Popovic, "Functional electrical stimulation therapy for restoration of motor function after spinal cord injury and stroke: A review," *BioMedical Engineering OnLine*, vol. 19, no. 1, p. 34, Aug. 2020, Accessed: 2026-01-26. DOI: 10.1186/s12938-020-00773-4. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/32819463/.

[12] N. Routledge, D. Zhang, and B. Metcalfe, "Electrical stimulation in upper limb assistance: Opportunities and challenges," *Frontiers in Neuroscience*, vol. 19, p. 1 702 889, Jan. 2026, Accessed: 2026-01-26. DOI: 10.3389/fnins.2025.1702889. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fnins.2025.1702889/full.

[13] O. A. Howlett, N. A. Lannin, L. Ada, and C. McKinstry, "Functional electrical stimulation improves activity after stroke: A systematic review with meta-analysis," *Archives of Physical Medicine and Rehabilitation*, vol. 96, no. 5, pp. 934–943, May 2015, Accessed: 2026-01-26. DOI: 10.1016/j.apmr.2015.01.013. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/25634620/.

[14] J. Eraifej, W. Clark, B. France, S. Desando, and D. Moore, "Effectiveness of upper limb functional electrical stimulation after stroke for the improvement of activities of daily living and motor function: A systematic review and meta-analysis," *Systematic Reviews*, vol. 6, no. 1, p. 40, Feb. 2017, Accessed: 2026-01-26. DOI: 10.1186/s13643-017-0435-5. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/28245858/.

[15] M. A. Khan, H. Fares, H. Ghayvat, *et al.*, "A systematic review on functional electrical stimulation based rehabilitation systems for upper limb post-stroke recovery," *Frontiers in Neurology*, vol. 14, p. 1 272 992, Dec. 2023, Accessed: 2026-01-26. DOI: 10.3389/fneur.2023.1272992. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/38145118/.

[16] J. S. Knutson, J. Chae, R. L. Hart, *et al.*, "Implanted neuroprosthesis for assisting arm and hand function after stroke: A case study," *Journal of Rehabilitation Research and Development*, vol. 49, no. 10, pp. 1505–1516, 2012, Accessed: 2026-01-26. DOI: 10.1682/jrrd.2011.09.0171. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/23516054/.

[17] K. H. Polasek, H. A. Hoyen, M. W. Keith, R. F. Kirsch, and D. J. Tyler, "Stability of the multi-contact nerve cuff interface for chronic nerve recording," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 17, no. 5, pp. 428–437, Oct. 2009, Accessed: 2026-01-26. DOI: 10.1109/TNSRE.2009.2032603. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/20096824/.

[18] H. Zhang, S. M. Sid'El Moctar, S. Boudaoud, and I. Rida, "A comprehensive review of semg-imu sensor fusion for upper limb movements pattern recognition," *Information Fusion*, vol. 125, p. 103 422, Jan. 2026. DOI: 10.1016/j.inffus.2025.103422. [Online]. Available: https://doi.org/10.1016/j.inffus.2025.103422.

[19] S. Xie, Y. Zhang, J. Li, Z. Wu, and Y. Bai, "Contralaterally controlled functional electrical stimulation for improving motor function after acquired brain injury: A systematic review and meta-analysis," *Archives of Physical Medicine and Rehabilitation*, vol. 105, no. 12, pp. 2327–2335, Dec. 2024, Accessed: 2026-01-26. DOI: 10.1016/j.apmr.2024.03.001. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/38490452/.

[20] J. S. Knutson, A. S. Friedl, K. M. Hansen, *et al.*, "Contralaterally controlled functional electrical stimulation for upper extremity recovery following stroke: A multisite randomized controlled trial," *Stroke*, Nov. 2025, Accessed: 2026-01-26. Online ahead of print. DOI: 10.1161/STROKEAHA.125.052891. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/41230603/.

[21] E. Ambrosini, G. Gasperini, J. Zajc, *et al.*, "A robotic system with emg-triggered functional eletrical stimulation for restoring arm functions in stroke survivors," *Neurorehabilitation and Neural Repair*, vol. 35, no. 4, pp. 334–345, Apr. 2021, Accessed: 2026-01-26. DOI: 10.1177/1545968321997769. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/33655789/.

[22] T. Shaowang, X. Liang, and S. Krishnan, "Sensor fusion on the edge: Initial experiments in the edgeserve system," in *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments (BiDEDE '22)*, New York, NY, USA: Association for Computing Machinery, 2022. DOI: 10.1145/3530050.3532924. [Online]. Available: https://doi.org/10.1145/3530050.3532924.

[23] S. Patel, H. Park, P. Bonato, L. Chan, and M. Rodgers, "A review of wearable sensors and systems with application in rehabilitation," *Journal of NeuroEngineering and Rehabilitation*, vol. 9, no. 1, p. 21, 2012. DOI: 10.1186/1743-0003-9-21. [Online]. Available: https://doi.org/10.1186/1743-0003-9-21.

[24] X. Wang, H. Yu, S. Kold, O. Rahbek, and S. Bai, "Wearable sensors for activity monitoring and motion control: A review," *Biomimetic Intelligence and Robotics*, vol. 3, no. 1, p. 100 089, 2023. DOI: 10.1016/j.birob.2023.100089. [Online]. Available: https://doi.org/10.1016/j.birob.2023.100089.

[25] J. Tosi, F. Taffoni, M. Santacatterina, R. Sannino, and D. Formica, "Performance evaluation of bluetooth low energy: A systematic review," *Sensors*, vol. 17, no. 12, p. 2898, 2017. DOI: 10.3390/s17122898. [Online]. Available: https://doi.org/10.3390/s17122898.

[26] R. Rondón, M. Gidlund, and K. Landernäs, "Evaluating bluetooth low energy suitability for time-critical industrial iot applications," *International Journal of Wireless Information Networks*, vol. 24, pp. 278–290, 2017. DOI: 10.1007/s10776-017-0357-0. [Online]. Available: https://doi.org/10.1007/s10776-017-0357-0.

[27] M. Spörk, C. A. Boano, and K. Römer, "Improving the timeliness of bluetooth low energy in noisy rf environments," in *Proceedings of the 2019 International Conference on Embedded Wireless Systems and Networks (EWSN 2019), Beijing, China, February 25–27, 2019*, Y. Liu, G. Xing, Y. He, and G. P. Picco, Eds., ACM, 2019, pp. 23–34. [Online]. Available: https://dl.acm.org/citation.cfm?id=3324324.

[28] D. B. de Oliveira, D. Casini, R. S. de Oliveira, and T. Cucinotta, "Demystifying the Real-Time Linux Scheduling Latency," in *32nd Euromicro Conference on Real-Time Systems (ECRTS 2020)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 165, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, 9:1–9:23. DOI: 10.4230/LIPIcs.ECRTS.2020.9. [Online]. Available: https://doi.org/10.4230/LIPIcs.ECRTS.2020.9.

[29] The Linux kernel community, *No_hz: Reducing scheduling-clock ticks*, Linux kernel documentation, Accessed 2025-12-25. [Online]. Available: https://docs.kernel.org/timers/no_hz.html.

[30] Z. Zhang, K. Yang, J. Qian, and L. Zhang, "Real-time surface emg pattern recognition for hand gestures based on an artificial neural network," *Sensors*, vol. 19, no. 14, p. 3170, 2019. DOI: 10.3390/s19143170. [Online]. Available: https://doi.org/10.3390/s19143170.

[31] I. Karacan, B. Topkara Arslan, A. Karaoglu, *et al.*, "Estimating and minimizing movement artifacts in surface electromyogram," *Journal of Electromyography and Kinesiology*, vol. 70, p. 102 778, 2023. DOI: 10.1016/j.jelekin.2023.102778.

[32] R. H. Chowdhury, M. B. I. Reaz, M. A. B. M. Ali, A. A. A. Bakar, K. Chellappan, and T. G. Chang, "Surface electromyography signal processing and classification techniques," *Sensors*, vol. 13, no. 9, pp. 12 431–12 466, 2013. DOI: 10.3390/s130912431.

[33] Applied Neural Research Corp., *M40 Muscle Sense: Wireless neuromuscular activity transmitter (product sheet)*, Product sheet, Document ID: A001S1M40A, PS-25-2, 2025. [Online]. Available: https://www.anrcorp.com/documents/M40_ProductSheet.pdf (visited on 12/26/2025).

[34] M. Straczkiewicz, N. W. Glynn, and J. Harezlak, "On placement, location and orientation of wrist-worn tri-axial accelerometers during free-living measurements," *Sensors*, vol. 19, no. 9, p. 2095, 2019. DOI: 10.3390/s19092095.

[35] *LSM6DSOX: 6-axis imu (inertial measurement unit) with embedded ai: Always-on 3-axis accelerometer and 3-axis gyroscope (datasheet)*, DS12814, Rev 4, STMicroelectronics, Jun. 2024. [Online]. Available: https://www.st.com/resource/en/datasheet/lsm6dsox.pdf (visited on 12/26/2025).

[36] gRPC Authors, *Introduction to grpc*, gRPC Documentation, Accessed: 2025-12-27. [Online]. Available: https://grpc.io/docs/what-is-grpc/introduction/.

[37] Google, *Overview*, Protocol Buffers Documentation, Accessed: 2025-12-27. [Online]. Available: https://protobuf.dev/overview/.

[38] M. Belshe, R. Peon, and M. Thomson, "Hypertext transfer protocol version 2 (http/2)," IETF, RFC 7540, 2015, Accessed: 2025-12-27. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7540.

[39] gRPC Authors, *Core concepts, architecture and lifecycle*, gRPC Documentation, Accessed: 2025-12-27. [Online]. Available: https://grpc.io/docs/what-is-grpc/core-concepts/.

[40] Microsoft. "Performance best practices with grpc." (2025), [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/grpc/performance?view=aspnetcore-10.0 (visited on 02/09/2026).

[41] C.-W. Lee and H.-M. Chuang, "Design of a seniors and alzheimer's disease caring service platform," *BMC Medical Informatics and Decision Making*, vol. 21, no. Suppl 10, p. 273, 2021. DOI: 10.1186/s12911-021-01626-3.

[42] S. P. Arjunan, D. K. Kumar, and G. Naik, "Computation and evaluation of features of surface electromyogram to identify the force of muscle contraction and muscle fatigue," *BioMed Research International*, p. 197 960, 2014. DOI: 10.1155/2014/197960.

[43] M. D. Ortigueira and J. T. Machado, "A review of sample and hold systems and design of a new fractional algorithm," *Applied Sciences*, vol. 10, no. 20, p. 7360, 2020. DOI: 10.3390/app10207360.

[44] X. Zhang, X. Chen, Y. Li, V. Lantz, K. Wang, and J. Yang, "A framework for hand gesture recognition based on accelerometer and emg sensors," *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans*, vol. 41, no. 6, pp. 1064–1076, 2011. DOI: 10.1109/TSMCA.2011.2116004.

[45] L. H. Smith, L. J. Hargrove, B. A. Lock, and T. A. Kuiken, "Determining the optimal window length for pattern recognition-based myoelectric control: Balancing the competing effects of classification error and controller delay," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 19, no. 2, pp. 186–192, 2011. DOI: 10.1109/TNSRE.2010.2100828.

[46] K. Englehart and B. Hudgins, "A robust, real-time control scheme for multifunction myoelectric control," *IEEE Transactions on Biomedical Engineering*, vol. 50, no. 7, pp. 848–854, 2003. DOI: 10.1109/TBME.2003.813539.

[47] L. Bao and S. S. Intille, "Activity recognition from user-annotated acceleration data," in *Proceedings of the 2nd International Conference on Pervasive Computing (PERVASIVE)*, 2004, pp. 1–17. DOI: 10.1007/978-3-540-24646-6_1.

[48] T. F. Chan, G. H. Golub, and R. J. LeVeque, "Updating formulae and a pairwise algorithm for computing sample variances," Stanford University, Tech. Rep., 1979.

[49] NIST/SEMATECH, *E-handbook of statistical methods: Interquartile range (iqr)*. [Online]. Available: https://www.itl.nist.gov/div898/handbook/prc/section1/prc16.htm (visited on 02/09/2026).

[50] L. Meng, J. Pang, Z. Wang, R. Xu, and D. Ming, "The role of surface electromyography in data fusion with inertial sensors to enhance locomotion recognition and prediction," *Sensors*, vol. 21, no. 18, p. 6291, 2021. DOI: 10.3390/s21186291. [Online]. Available: https://doi.org/10.3390/s21186291.

[51] Bluetooth SIG, *Bluetooth core specification v5.4: Part g — generic attribute profile (gatt)*, Bluetooth SIG Specification (HTML), Accessed: 2025-12-29. [Online]. Available: https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Core-54/out/en/host/generic-attribute-profile--gatt-.html.

[52] C. Gomez, J. Oller, and J. Paradells, "Overview and evaluation of bluetooth low energy: An emerging low-power wireless technology," *Sensors*, vol. 12, no. 9, pp. 11 734–11 753, 2012. DOI: 10.3390/s120911734. [Online]. Available: https://doi.org/10.3390/s120911734.

[53] V. V. Tipparaju, K. R. Mallires, D. Wang, F. Tsow, and X. Xian, "Mitigation of data packet loss in bluetooth low energy-based wearable healthcare ecosystem," *Biosensors*, vol. 11, no. 10, p. 350, 2021. DOI: 10.3390/bios11100350. [Online]. Available: https://doi.org/10.3390/bios11100350.

[54] A. Asili, "Minimizing data loss in bluetooth low energy (ble): A parallel transmission and application-layer modification approach," Accessed: 2025-12-29, Bachelor's thesis, Blekinge Institute of Technology, 2025. [Online]. Available: https://www.diva-portal.org/smash/record.jsf?pid=diva2:1973567.

[55] Google, *Language guide (proto3): Protocol buffers documentation*, protobuf.dev, Accessed: 2025-12-29. [Online]. Available: https://protobuf.dev/programming-guides/proto3/.

[56] IBM, *What is grpc?* IBM Think, Accessed: 2025-12-27. [Online]. Available: https://www.ibm.com/think/topics/grpc.

[57] L. Chaari Fourati and S. Said, "Remote health monitoring systems based on bluetooth low energy (ble) communication systems," in *The Impact of Digital Technologies on Public Health in Developed and Developing Countries*, M. Jmaiel, M. Mokhtari, B. Abdulrazak, H. Aloulou, and S. Kallel, Eds., Springer, 2020. DOI: 10.1007/978-3-030-51517-1_4. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-51517-1_4.

[58] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Computing Surveys*, vol. 35, no. 2, pp. 114–131, 2003. DOI: 10.1145/857076.857078.

[59] A. Ahmad. "System design interview gold: From oom errors to stable architectures." Accessed: 2026-01-08. See section "Dropping (Load Shedding)" for "Drop Oldest", System Design Nuggets (DesignGurus) on Substack. (Jan. 2026), [Online]. Available: https://designgurus.substack.com/p/understanding-backpressure-mechanism.

[60] PLUX Wireless Biosignals S.A. "Muscleban ble datasheet." Last document update: May 21, 2024. Accessed 2026-02-18. (May 2024), [Online]. Available: https://support.pluxbiosignals.com/wp-content/uploads/2024/05/muscleBAN_BLE_Datasheet.pdf.

Appendix A

Supplementary Material

This appendix contains additional material that complements the main results but would be too detailed for the core chapters.

A.1 Implant unit

Figure A.1 shows the implant unit. It is responsible for measurement, stimulation, and communication with the external unit. Measurement data (neuronal signals) and implant status information (e.g., temperature and impedance) are transmitted to the external unit and forwarded to the computer system. Conversely, measurement and stimulation commands are sent from the external side to the implant. The implant contains the required electronics and firmware for these functions. The implant is powered inductively via a coil. For biocompatibility, the electronics are hermetically encapsulated in a ceramic housing. The ceramic capsule and the coil with magnet are embedded in medical-grade silicone, forming a biocompatible package. In addition, the implant includes a ground electrode that serves as a reference for measurements.

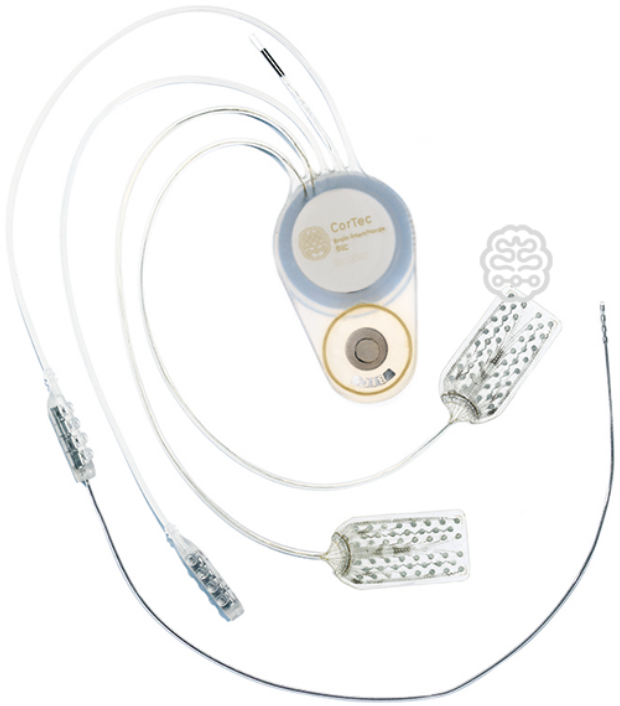


Figure A.1: Implant unit with connected multi-electrode arrays of the CorTec Brain Interchange System.